

Copyright © by Michael Hegemann

This documentation is regularly updated for the latest plugin version.

Binary Plugin Version [Launcher]
1.85 (UE5.4 - UE5.8)

Source Plugin Version [Discord]
1.86 (UE5.4 - UE5.9)

Example Project Version
2.8 (UE5.5+)

EMS Addons Version
1.1.2 (UE5.8+)

Previous plugin and example project versions are compatible with older UE builds.
It's recommended to use the latest Plugin version. Everything is backwards compatible.

Chapter 1. Introduction

1.1. General Concepts

Easy Multi Save (EMS) is a comprehensive solution designed to simplify the saving and loading process for your projects. Its flexible structure ensures ease of use, even for those with only basic Unreal Engine knowledge. EMS is compatible with all game types and platforms, focusing on the **efficient** and **reliable** management of complex data.

Save/Load:

- Data to an infinite amount of save game slots.
- Player/Client including Controller, Pawn and State.
- Level and Sub-Level Blueprints.
- Open worlds with World Partition or Streaming.
- Actors placed in the level or spawned at runtime.
- Actor components for all Actors.
- Blueprint Structs that can be recursive.
- UObjects as part of an Actor or to a custom file.
- Custom Save Objects, for storing settings or additional data like progress.

The system is entirely initialization-free, eliminating the need for manual creation of save game objects or actor Ids. Everything is managed automatically to ensure a hassle-free experience.
EMS is built with a clear and consistent structure, embodying the core Unreal Engine philosophy:
"Work smarter, not harder."

1.2. Installation

The installation works as with every other UE Plugin. After downloading, look up the Easy Multi Save Plugin in **Library ➔ Vault** . Then click on **Install in Engine** . Now you are ready to go.

1.3. Example Project

The Example Project shows most examples from here in action.

Download

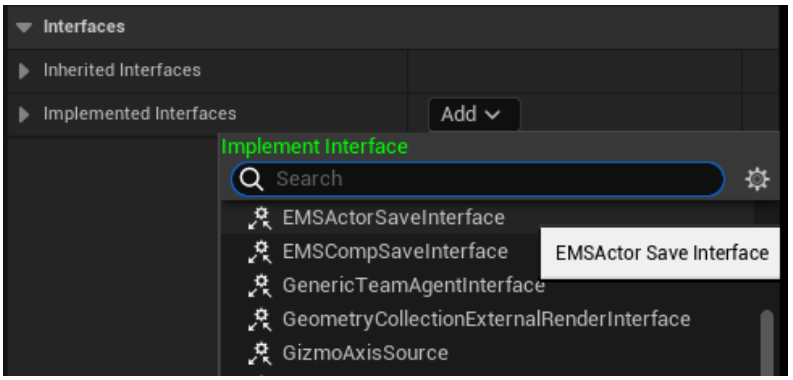
It only features a minimal amount of content, so you should be able to copy and paste things into your own projects. The EMS_HelperFunctions library also features a few useful methods for you to use.

1.4. Quick Start

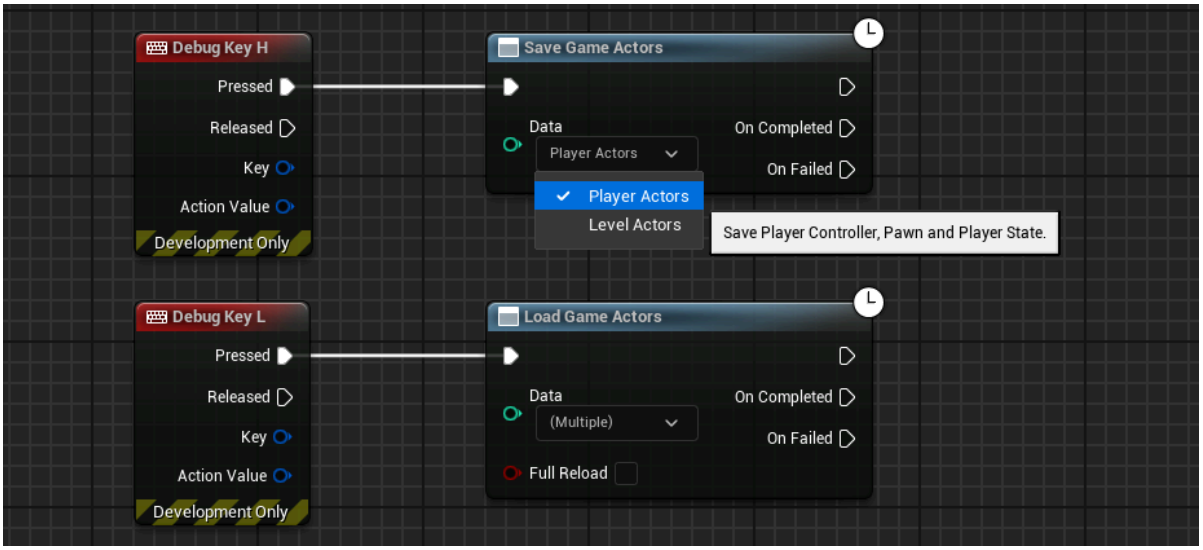
While the plugin offers deep functionality, everything is opt-in. You can get started right away, then use more advanced features later. To get up and running:

➤ Add the EMS Actor Save Interface to your Character Blueprint.

➤ Add the EMS Actor Save Interface to one of your Actor Blueprints or a Level Blueprint.



➤ Call the Save and Load Game Actors nodes by simply binding them to debug keys. Make sure to check Level and Player Actors on both nodes.



This is the simplest way of saving and loading Actors, you are now free to expand upon this and add variables to your classes etc. Make sure all variables you want to save have the Save Game flag checked.

Chapter 2. Reference

2.1. File System

By default, EMS uses a multi file saving structure, where all files are put into a single folder. This also supports easy sharing of save games between players, and you are able to just upload the files you need to services like Steam Cloud or Amazon S3.

📁 **File Structure**

/SaveGames/SaveGameName/
The folder holds all the data for a single save game / save slot.

/SaveGameName/Slot.sav

Holds information on the save slot, like level name, list of players and timestamp.
Slot info is always saved automatically when calling one of the save functions.

/SaveGameName/Level.sav
This file holds all placed and runtime spawned level actors with the save interface.

/SaveGameName/Player.sav
This file contains all the information on the player controller, pawn and player state.

/SaveGameName/Thumb.sav
This is the thumbnail created for the save game from a 2D rendertarget source.

/SaveGameName/*Filename.sav
A custom save file with the desired name.
Used by Custom Save Objects, Custom Player Data or Object Collections.

📁 **File Locations**

For an editor or development packaged game, the save files are stored in:
/ProjectName/Saved/SaveGames

For a shipping packaged game on Windows, the save files are typically stored in:
Users/.../AppData/Local/ProjectName/Saved/SaveGames

📁 **Console File System**

In the plugin settings, you can change the file system from Desktop to Console. In this case, EMS does not create separate folders for save files, but attaches the slot name to the appropriate file. Such as /Saved/SaveGames/MySaveGame_Player.sav

2.2. Blueprint Node Overview

Less complexity, more power. Here you will get an overview of each function and what it does. Make sure to check out the EMS_FunctionOverview in the Example Project.

Do not use Unreal default SaveGame nodes to read or write EMS objects or files. Independent save systems may coexist with EMS.



⤶ ↳ Game Actor Functions

Save Game Actors
The main function for saving Game Actors.

> ↳ Data
> Check what data you want saved. Level and/or Player.

Load Game Actors
The main function for loading Game Actors.

> ↳ Data
> Check what data you want loaded. Level and/or Player.
>
> ↳ Full Reload
> If false, load only Actors that have not been loaded.

⤶ ↳ Save Slot Functions

Set Current Save Slot
Set the name of the current save game slot for all save functions. See Save Slots.

> ↳ Save Game Name
> The current slot will be set to this name.

Get All Save Slots
Returns the array of all available save game slots.

↵ Return Value
The array of save game slots, sorted by their time of saving.

Get Current Save Slot
Get data from the current save game slot.

↵ Save Game Name
Returns the name of the current save slot. Simply a shortcut for convenience.

↵ Return Value
The slot info save object.
From this you can access the Slot Info struct to get the level and timestamp etc.

Get Named Save Slot
With this node, you can get data for a specific save game slot.

↳ Save Game Name
The input slot name you want to get the slot object for.

↵ Return Value
The desired slot info save object.

Delete Save Slot
This function deletes a save game and all its files, or only specific files.
Make sure that the input is not empty!

↳ Save Game Name
The slot/folder to be deleted. Should never be empty.

↳ Data
The specific data you want to delete from a slot.

Does Save Slot Exist
Checks to see if a specific slot does exist.

↳ Save Game Name
The slot name you want to check.

↳ Complete
Will check for Level, Player and Slot instead of just the Slot file.

↵ Return Value
True if the save slot was found.

⤶ ⤷ Save User Functions

Set Current Save User
Sets the current save user. Useful if you have a multi-user game.
Stores data under Saved/SaveGames/Users/<UserName>/ .
Clearing the Save User returns EMS to the default Saved/SaveGames/ location.

> ⤷ User Name
> The name of the current save user. Cleared if none.

Delete Save User
This function deletes a save game user folder and all of its files.
Make sure that the input is not empty!

> ⤷ User Name
> The name of the save user to delete. Should never be empty.

Get Current Save User
Gets the current Save User name.

> ⤶ Return Value
> The current Save User.

Get Save Users
Returns all the available save game users.

> ⤶ Return Value
> The array of all available save game users, sorted by name.

⤶ ⤷ Custom Save Functions

Get Custom Save Object
This will return and cache a custom save object, deriving from EmsCustomSaveGame.
It can be used for any kind of data across all of your Blueprints.
See Custom Save Objects.

> ⤷ Save Game Class
> The input class based off EmsCustomSaveGame.

> ⤷ Save Slot
> Optional. Access a Custom Save Object from a specific slot.
> If empty, it will use the current save slot.

> ⤷ File Name
> Optional. Override the default file name from the Blueprint.
> Allows to have multiple Custom Save Objects with the same class.

↩ Return Value
The returned custom save object, auto-cast to the input class.

Save Custom Object(s)
Saves all cached custom save objects automatically.
Optionally, it can save one particular custom save object.

↳ Save Game
In case you only want to save one specific Custom Save Object.

↩ Return Value
If the save was successful.

Delete Custom Save Object
Deletes the desired input Custom Save Object and its File.

↳ Save Game
The custom save object you want to delete. From Get Custom Save Object.

Does Custom Save Exist
Checks to see if the Custom Save File for the input actually exists.

↳ Save Slot
Optional. The slot you want to check the file in.

↳ File Name
The custom save file you want to check.

Reset Custom Save Object
Resets a custom save object either in memory or to its class default values.
Use carefully. Resetting values right before saving can overwrite important data on disk.

↳ Type
Defines the type of reset operation to perform. You can reset the object to its default values, clear its in-memory cache, or perform both actions at once.

↩ ↳ Streaming Related Functions

Is Level Streaming Active
If any Level Streaming is still active from the perspective of EMS.
This can be used to defer or prevent saving and loading during streaming.
It should be used when the log issues a warning about this.

↩ Return Value
If a level streaming process is currently active.

Is World Partition Save
Checks if the current persistent level is a partitioned World from the perspective of EMS.

↵ Return Value
If the current persistent level uses World Partition with EMS.

Clear World Partition Save
Clears all cached World Partition save data. For example, when teleporting the player, you would clear the data and then manually load level actors after the location change.

↵ ↳ Multi-Level Related Functions

Clear Multi-Level Save
When using Multi-Level Saving, this will remove all save data from currently unloaded levels.
This can be used when switching slots or users.

↵ ↳ Raw Object Functions

Save Raw Object Data
Saves any valid UObject as part of an Actor. See Raw Object Saving.

↳ Data
A struct consisting of the UObject to save and its unique Id.

Load Raw Object Data
Loads any valid UObject as part of an Actor.

↳ Data
A struct consisting of the UObject to load and its unique Id.

↵ ↳ Data Helper Functions

Set Actor Save Properties
Change Actor specific save and load properties. See Save Properties.

Is Loaded State
Returns true if an Actor was loaded by the system.

↵ Return Value
True if the Actor was loaded.

Is Saving Or Loading
Returns true if Save Game Actors or Load Game Actors is currently active.

> ⤶ Return Value
> True if one of the async save or load nodes is currently active.

Wait for Load Actors Completed
Explicitly waits for a Load Game Actors operation.
It can only be triggered once per execution. This is useful if you want to use loaded variables in your BeginPlay event for example. Or wait for a saved reference to another Actor to be valid. You can either wait for the full operation to be complete, or check if a single Actor has completed loading.
*Previous to v1.83, this node was known as 'Wait For Save or Load Completed'.

> ⤷ Mode
> The load data to wait for, either Player, Level or Both.

> ⤷ Check Actor
> If set, this will only check if one particular Actor has been loaded.
> This overrides the Mode input.

⤶ ⤷ Thumbnail Functions

Export Save Thumbnail
Export a save game thumbnail image from a rendertarget source.

> ⤷ Texture Render Target
> The render target source from which the thumbnail is generated.

> ⤷ Save Game Name
> The name of the save game slot, for which the thumbnail should be exported.
> Usually this should be the current slot, but there may be cases when you want to export for a different one.

Import Save Thumbnail
Import a previously exported thumbnail image.

> ⤷ Save Game Name
> The name of the save game slot, from which the image should be imported.

> ⤶ Return Value
> The thumbnail image, imported as Texture2D.

⤶ ⤷ File Integrity Functions

Check Save File Integrity
Check the integrity of available files. See File Integrity.

↩ Check Type
The type of file to check.

↩ Custom Save Name
When checking a Custom Save or Custom Player, this is the file name either within the current slot or independent of it.

Get Backup Name
Get the name of the backup save slot. See Automatic Backup.
Can be used in all Save Game Name related inputs.

↳ Base Name
The slot name you want to get the backup for.

↩ Return Value
The backup name of the slot.

Get Save Game Version
Returns information about the user defined Save Game Version.

↩ Save File Version
The actual Save Game Version from the last loaded save files.

↩ Settings Version
The current Save Game Version defined in the settings.

↩ ↳ Custom Player Functions

Save Custom Player
Saves Controller, Pawn and PlayerState to a dedicated save file.
This can be used to save individual players on the server.
See Multiplayer.

↳ Controller
The Controller associated with the Pawn and PlayerState.

↳ File Name
The name of the actual File. This should be a Unique Player ID.

↩ Return Value
Indicates if the Custom Player was saved successfully.

Load Custom Player
Load Controller, Pawn and PlayerState from its dedicated save file.

↳ Controller

The Controller associated with the Pawn and PlayerState.

↳ File Name
The name of the actual File. This should be a Unique Player ID.

⤶ Return Value
If the Custom Player was loaded successfully.

Delete Custom Player
Will delete a Custom Player file.

↳ File Name
The name of the actual File to be deleted.

⤶ Return Value
If the Custom Player was deleted successfully.

2.3. Saving and Loading

Actors
All Actors implementing the Save Interface are considered for saving and loading.
Saved runtime Actors are automatically respawned into the persistent level on load.
The plugin also handles destruction for all placed Actors with the Save Interface automatically.
You can turn this off in the Plugin Settings, in case you want to handle it manually.

Actor Variables
Just check the Save Game property in the details panel and it will automatically save any variable as part of the Actor implementing the Save Interface.



Transforms and Attachments
Actor transforms are automatically saved/loaded by these rules.

Actor Type	Mobility	Attach Parent	Transform Saved
Runtime	Any	Any	Yes

Placed	Movable	No	Yes
Placed	Any	Yes	No
Placed	Static/Stationary	No	No

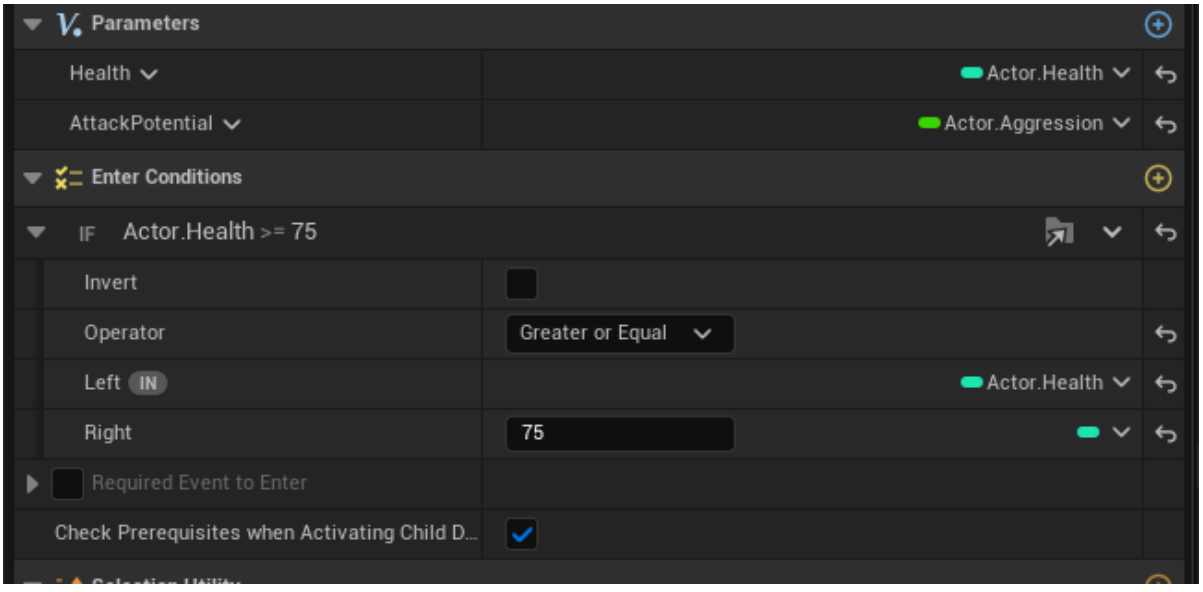
For physical Actors, the simulated component must be the root.
For Components, the relative Transform is saved.
The Pawn automatically saves/loads location and rotation.
The Controller automatically saves/loads the view rotation.

When saving a level-placed Actor that is attached to another, its transform will not be saved.
For runtime spawned Actors, you can manually handle attachments from the Actor Loaded Event.
Of course, you can always save a custom Transform and override the behavior.

Blueprint Structures
By default, a struct with Save Game checked has all of its elements automatically saved. You can turn this off in the Plugin Settings, so only Save Game flag variables are considered.
Maps, Sets and all Array types also work this way.
Instanced Structs are also fully supported, but always require the Save Game flag for individual variables.

NPCs
Saving AI Controllers for level placed AI with auto-possess works out of the box. It is generally recommended to only save the Pawn and not the Controller. For spawned AI, it requires a bit more work. The example project comes with an AI example level showing how to save spawned AI Pawns.

State Tree
For State Trees, you can simply bind saved Actor properties from the Context Actor to State Parameters. Such as Health, Ammo etc. and then let the tree drive its state by those properties.
It is recommended to start the tree logic manually after loading.



UObjects
For UI Widgets or other UObject based classes, you can use Raw Object Saving.
In case you require a fully custom setup for UObjects, look into Object Collections.

Single Actor Instances
To save and load one or more specific Actor instances rather than all eligible Actors, Object Collections do also offer a simple and generic solution for this.

Game Instance
Usually, the Game Instance is just used to pass variables from one level to another and not for saved data. Using Custom Save Objects is a lot easier and more flexible. But in case you require to save the Game Instance, you can just use Object Collections as well.

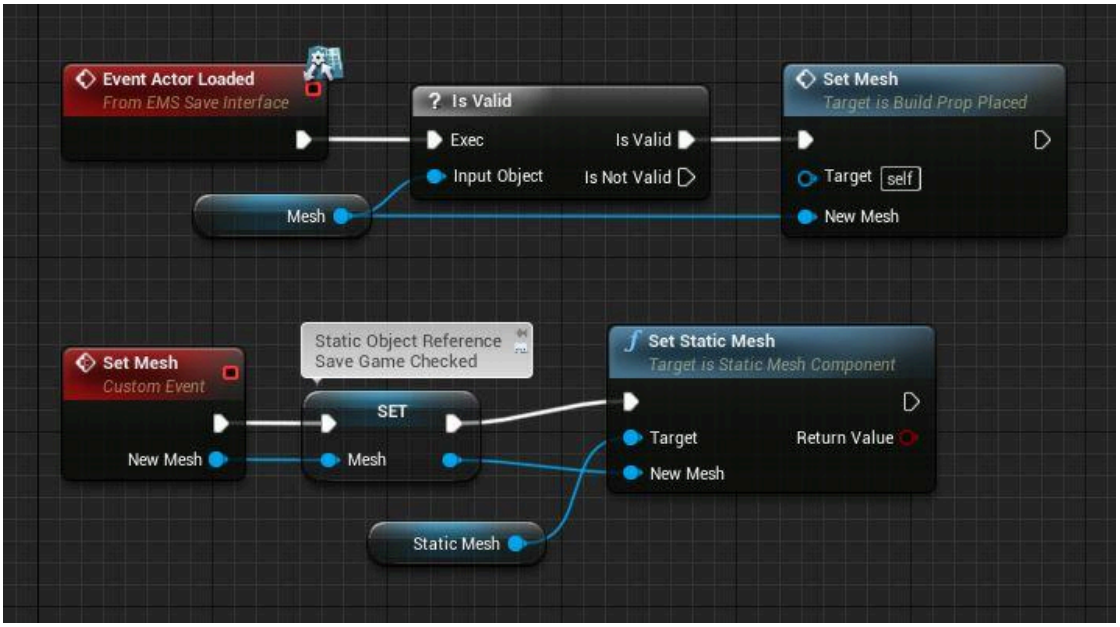
Level Instances
Saving and Loading Actors within placed Level Instances is fully supported.
See Level Streaming for further information.

Instanced Static Meshes
To persist instances of a HISM component, store each instance index and transform in a Blueprint struct on the owning Actor. Then recreate the instances from this data in the Actor Loaded event. You may also include other instance specific data in the struct.

Object References
Saving direct Object References works for simple cases, like storing a reference to a random Level Actor. However, it's generally discouraged, as the reference must be valid at the frame of loading.
By default, Unreal can only return the resolved object property type. Instead, prefer using Soft Object references, which can be resolved at any point after loading and also used with Multi-Thread saving.
Another culprit are Blueprint Structs that contain direct references. Due to an engine limitation, Unreal may break the struct completely when the reference is invalid. Soft Object references will always work in this case as well.

Soft Object References
Soft Object references are fully supported for any case. This works for runtime Actors respawned by EMS and level-placed Actors. For saving and loading, you will just need to resolve from/to the Soft Object and use it like any other variable. For example, you have a character and drop a weapon in the level, the weapon has the EMS Interface added and is respawned on load. Now, in your character, you can add a Soft Object Reference to the dropped weapon and access it. You just need to make sure that the weapon has already respawned when you use the reference.
This can be achieved by using the Wait For Load Actors Completed node.

Asset References
Variables referencing Meshes, Materials or Anim Instances can be saved, as long as it contains a direct reference to an asset. For example, you have an Actor and you want to set its Static Mesh Component to a new Mesh. This also works with Data Assets.

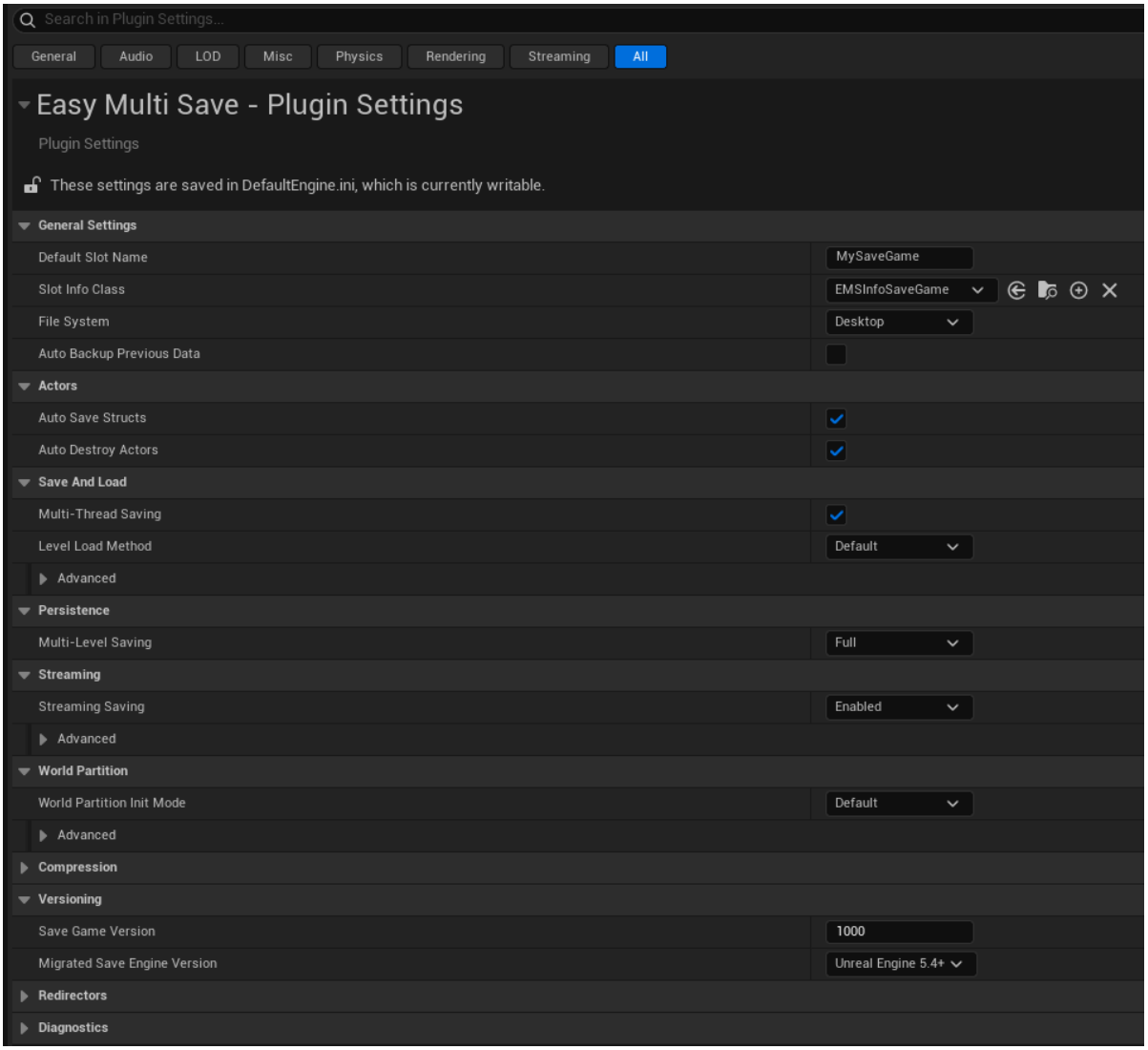


Auto Saving
To add automatic or event-driven saving to your project, simply call the Save Game Actors and/or Save Custom Objects functions from a looping timer or in response to a trigger event, such as a level-placed Trigger as you see fit. No additional setup is required.

2.4. Plugin Settings

First, it's important to know that you don't need to change any settings to get going. But it's good to know what they do. To change settings, go to:

Project Settings ➔ Easy Multi Save ➔ Plugin Settings
For versions earlier than v1.85: **Project Settings ➔ Plugins ➔ Easy Multi Save**



↓ General Settings

Default Slot Name
If you don't specify a current slot name, or if you just want to use a single save game without slots, this will be the name of your save game.

Slot Info Class
If you want to use custom data for your Save Slots, this defines your custom Blueprint class for it. Usually you are fine with the default Save Slot class. See Custom Slot info.

File System
Select the file system to use. Either Desktop or Console. If you are creating a console game, you must use the Console file system. For all other platforms, you can use any of them. See File System.

Auto Backup Previous Data
Automatically creates a backup of the previous save data. See Automatic Backup.

↓ Actors

Auto Save Structs
Structs are saved automatically for all Actors like any other variable.

Otherwise, only the 'Save Game' checked variables inside the struct asset are saved.

Auto Destroy Actors
Enable automatic saving and loading of destroyed Actors, re-destroying them upon load.
Handles all Actors that implement the Actor Save Interface.

↓ Save and Load

Multi-Thread Saving
When enabled, Actor saving is processed on background threads.
This is useful if your levels contain many Actors with complex data, as it helps prevent hitches during gameplay. Exiting or switching levels during background save, will preventively cancel the operation and not (over)write any data.

Level Load Method
The method to use for loading Level Actors. See Load Methods .

→ Save and Load - Advanced

Load Batch Size
The number of Actors to load in one batch for deferred or multi-thread loading.

Async Wait Delay
How long the Async nodes are allowed to wait. For example, when calling Load Game Actors, the node will wait for an initialized World or valid Pawn until it fails.
The default is 10 seconds, with a minimum of 5 seconds.

Pre-Save On Game Thread
The Actor Pre-Save event is forced to be called on the game thread when using Multi-Thread saving. This should be enabled if you access non-thread-safe Game Objects such as Timers or Rendering related properties directly from the event. Saving may take longer, but remaining save processing continues in the background.

↓ Persistence

Multi-Level Saving
The method to use for keeping data between levels. See Multi Level Saving.
Required to save Streaming and World Partition Levels.

↓ Streaming

Streaming Saving
Automatically stores streaming Sub-Level and World Partition data in memory.
It is enabled by default and requires Multi-Level saving. Saving to disk is done as usual.
See Level Streaming.

Streaming Save Threshold

Minimum duration a streaming level must remain unloaded before its automatic memory save is processed. This batches rapid level-unload operations and avoids unnecessary work when a level immediately streams back in.

→ **Streaming - Advanced**

Allow Dynamic Streaming Levels
Enables support for Actors within Level Instances in Sub-Levels, Runtime spawned Level Instances and multiple Streaming Levels of the same type. This setting is not backwards-compatible, as it generates different unique IDs for Actors.

↓ **World Partition**

World Partition Init Mode
You can choose to automatically load level Actors at the beginning or do it manually.

→ **World Partition - Advanced**

World Partition Init Delay
The time it takes to prepare until the initial automatic load of the World Partition level starts. During this time, you need to position your player either manually or by loading Player Actors. Since the player's camera acts as a streaming source, it will determine the initial area and Actors that are streamed in and also loaded by EMS.

World Partition Init Threshold
The minimum time that level streaming must remain in a finished state until the initial World Partition load will start. Due to async level streaming, there are several frames in between different cells, where the Unreal Engine thinks that streaming is finished, even if it isn't. To prevent loading too early, EMS will wait until it's actually completed.

↓ **Compression**

Thumbnail Image Format
Image format of saved thumbnails.

↓ **Versioning**

Save Game Version
A predefined game version that can be compared against files from the current slot. Using the Check Save File Integrity node.

Migrated Save Engine Version
See Versioning.

↓ **Redirectors**

Runtime Actor Redirects
Level Redirects

See Data Redirectors

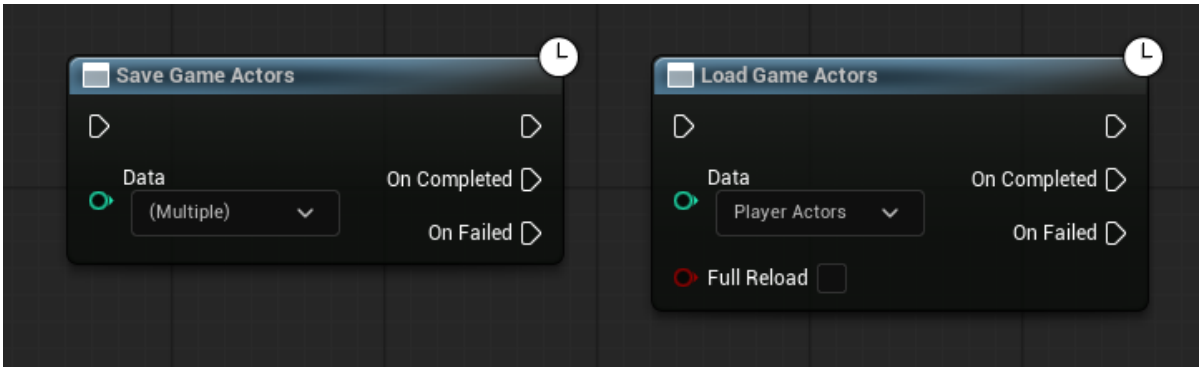
↓ **Diagnostics**

Advanced Save Report
Log Object Data Size
See Debugging and Testing

Chapter 3. Basic Use

3.1. Game Actors

Save Game Actors and Load Game Actors are the main functions for saving/loading your game world.
They both work similarly and will handle every Actor with the Save Interface.



Data Input
You can check what data you want included. Level and/or Player.
For Player Actors, the nodes wait for a valid Pawn.
For Level Actors, the nodes wait for a valid and initialized World.
If loading both, it will not start until it has a valid Pawn.
For more detailed information, see Player Data and Level Data.

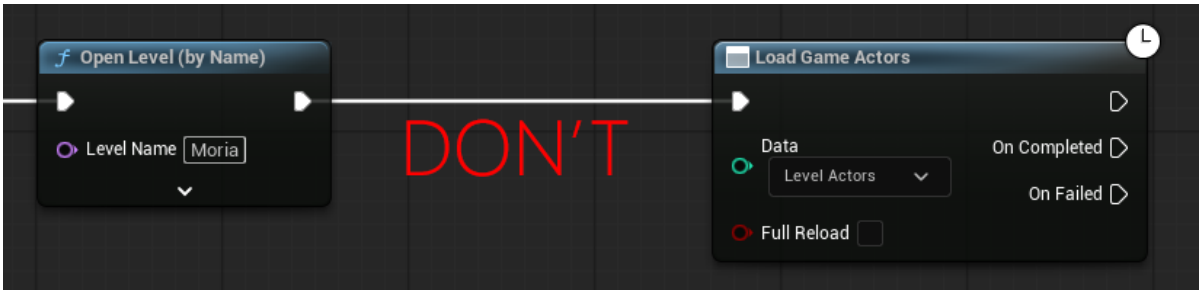
Save Game Actors
This will write Actor save data to disk for the current save slot. It can be called from anywhere at any time. You can call it from the UI, a quick-save key or even with a timer or trigger. It can also be called while the game is paused if that is required.

Load Game Actors
One call to Load Game Actors should always be issued on game start to load data into memory and properly restore saved runtime Actors. Unless you are in a World Partition level, which can handle the saved Level Data automatically on initialization.

For a single player game, the *Possessed* event of your character and the *BeginPlay* or *OnRestartPlayer* events in the Game Mode are recommended. *BeginPlay* in the Level Blueprint will work as well.

Check out the example project to see a possible setup.

Avoid calling it after Open Level in the same graph, since it will never be executed.



Full Reload
The Load Game Actors node has a 'Full Reload' checkbox. This will allow you to reload already loaded Actors without the need to restart the level. If unchecked, only Actors that were not previously loaded are considered.

Concurrency
Only one call with the same data may be active at a time. One with Level and one with Player data. But you cannot make two calls with Level data for example. Obviously, load and save cannot be active at the same time.

3.2. Save Slots

If you only want to use a single save game, you don't need the save slot system at all. EMS supports single slot, static slots and an infinite number of dynamic slots.

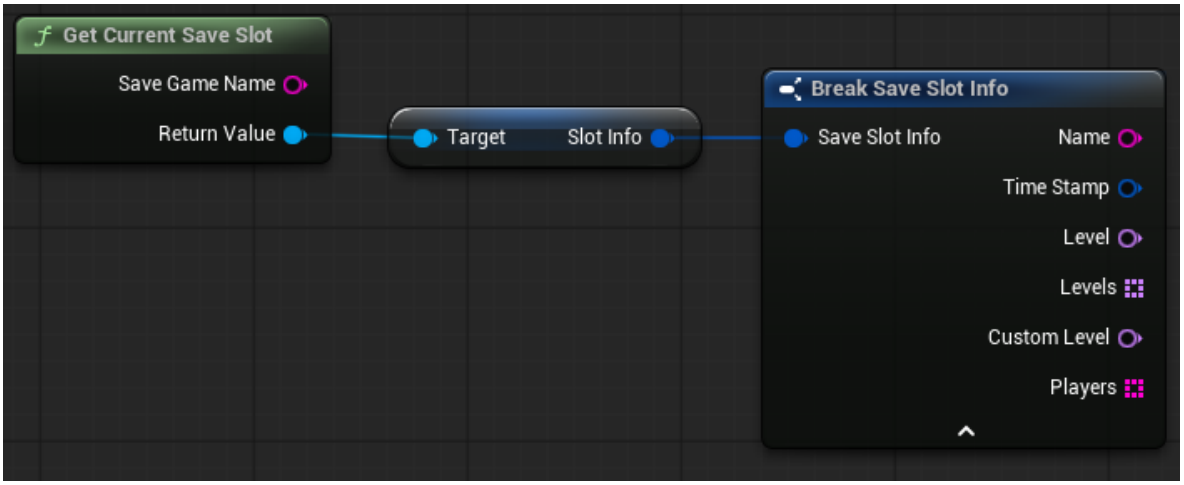
Calling Set Current Save Slot will set the global slot name for all EMS functions.
The new slot will be automatically created once a save function is called.



A slot in EMS is simply a string, nothing more. The setup is handled automatically.
If no slot name is set, it will use the Default Save Slot Name from the Plugin Settings.

To start a new game, you can just set a temporary save slot. Depending on your project, you may also call Clear Multi Level Save and Clear World Partition Save to empty the relevant caches.

The save slot also stores important meta-data, that you can access like this:

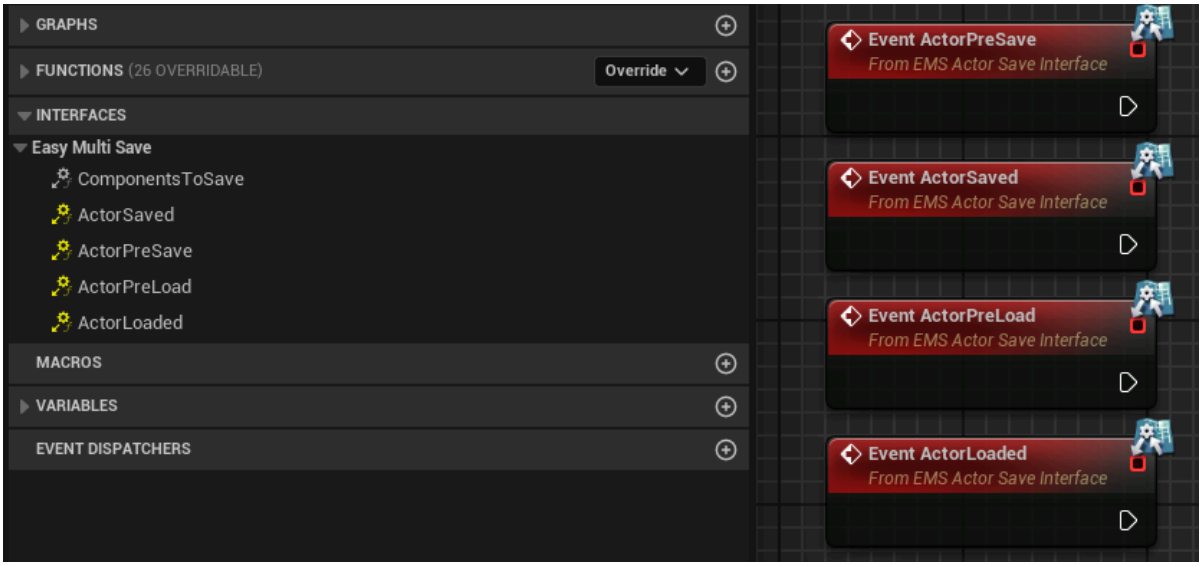


All data inside a save slot can be deleted using the Delete Save Slot node. You can remove only Player or Level Data from a slot. This is especially useful for Roguelite games, where you sometimes want to only delete Level Data after a failed run.

More topics about save slots can be found in the Custom Slot Info and Additional Data sections.

3.3. Save Interface

Every Actor you want to save, needs the EMS Actor Save Interface added to it.
The Interface calls some useful functions and events.



ActorPreSave
Called before saving the Actor, here you can safely set all kinds of variables and make sure they have the exact value you want at the time of saving.

ActorSaved
Called after saving the Actor and its components have finished.

ActorPreLoad
Called right before the Actor and all of its components are loaded.

For runtime spawned Actors, this is also called before the construction script.
It can be useful for disabling physics before loading and for special cases, such as preserving a default value. But most of the time, you won't need it.

ActorLoaded
This event is called after loading the Actor and its components have finished.
From here on, it's safe to access all the loaded variables of the Actor.
It is the most commonly used event.

ComponentsToSave
Here you can add an array of Components that you want to save.
This function has no effect for the level blueprint, since it cannot have components.
See Actor Components .

3.4. Level Data

Level Actors, Game Mode and Game State are saved to the Level file.
You can save placed as well as runtime spawned Actors and mix them up, even if they share the same class. Level Blueprints and Sub-Level Blueprints just work like every other Actor in the Level.

Destroy and Reload
When you destroy spawned Actors and perform a 'Full Reload' without restarting the level, you will need to clean up old references. Call Collect Garbage before running Load Game Actors. If you re-open your persistent level, this step is not required.

Procedural Levels
For a procedural generated level, you would save the Actors directly and then load the level from the saved file, instead of generating it. You can also combine this with level streaming and saving/loading sublevels. If your level has thousands of Actors, you should definitely use one of the advanced Load Methods .

3.5. Player Data

The Player Controller, Pawn and Player State are saved to the Player file.
Load Game Actors for the player will wait for a valid Pawn before it proceeds.
When loading level and player at once, the player Actors are loaded first.

Inventory
To save complex inventory, it's recommended to store the data inside your Pawn or a Component. Use a struct to store all inventory variables and create/spawn the inventory on load from the saved data.

Persistent Player
To transfer player data between levels, just use Multi Level Saving.

Multiplayer

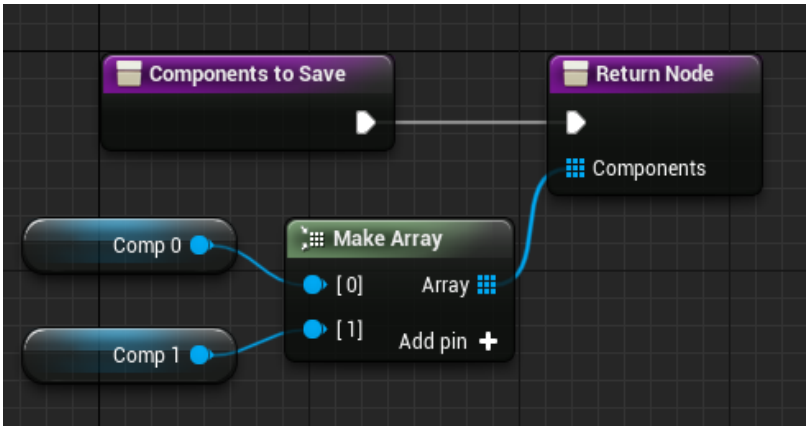
You can either save the local client in a P2P multiplayer game or use server side saving for all Player Actors with the Custom Player nodes. See Multiplayer.

3.6. Actor Components

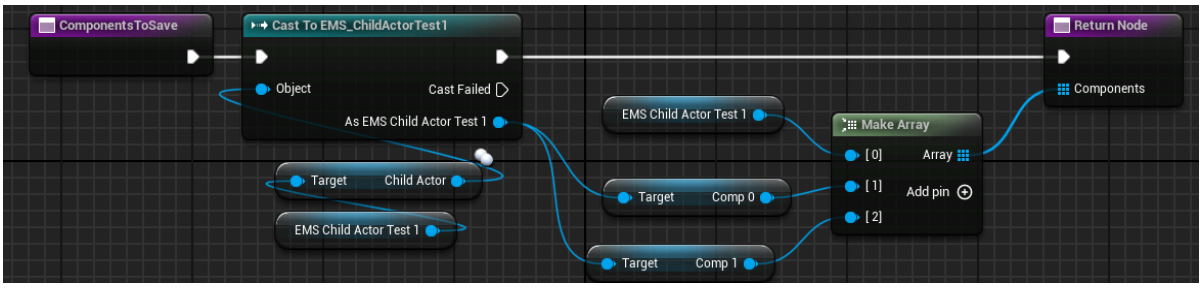
Component Interface
The EMS Component Save Interface can be added to any Custom Component, which will save it as part of an Actor that contains the EMS Actor Save Interface. This is compatible with Components from a Modular Game Feature. Components can also include their own Events like Actors. Saving works for all Custom Components and their data.



Component Array
Components can also be added to an array, as part of the EMS Actor Save Interface.
In this case, Components will not get any custom events and don't need their interface.



The array can also process Components spawned at runtime.
Just make sure you input a valid Reference to the added Component.
You can also save Components from another Actor like this:



Child Actor Components
For Child Actors, you must use the Array and not the Component Save Interface. If the Actor Class of the Child Actor has the Actor Save Interface, it will not get saved as a Component and log a warning.

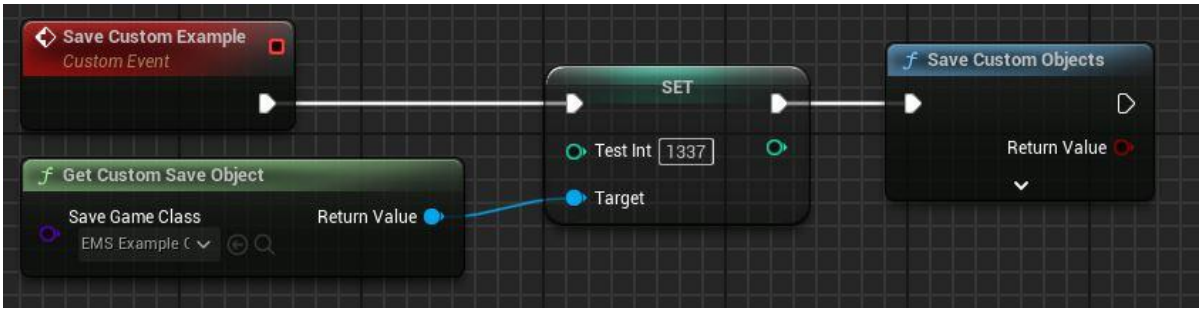
3.7. Custom Save Objects

A flexible system to store custom data similar to the UE default save, but much more efficient to use. Custom Save Objects are fully Independent of Actor saving.
The interface(s) or events are not required for this.
Variables also don't need the Save Game property flag, since the complete object is saved.

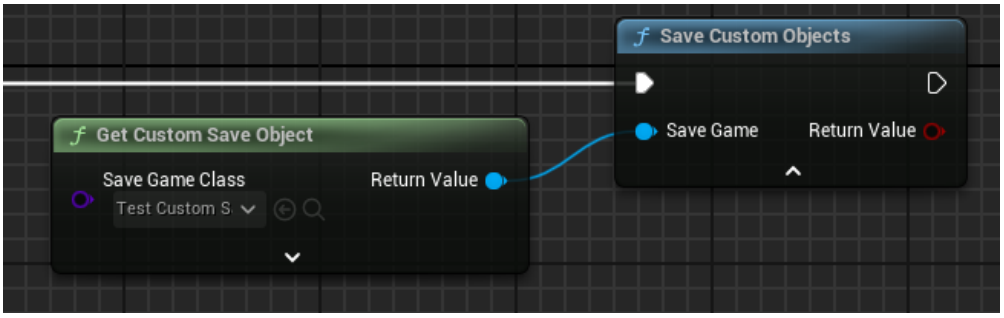
You create a child Blueprint of EmsCustomSaveGame and add whatever variables you like. This can be used to save UI related data for widgets or settings. It can also be handy for storing player stats and game progress. There are no limits to the number of Custom Save Objects in your project.

Basic Use
Using custom saves is very easy. It only requires two nodes from the plugin.
Get Custom Save Object and Save Custom Objects.

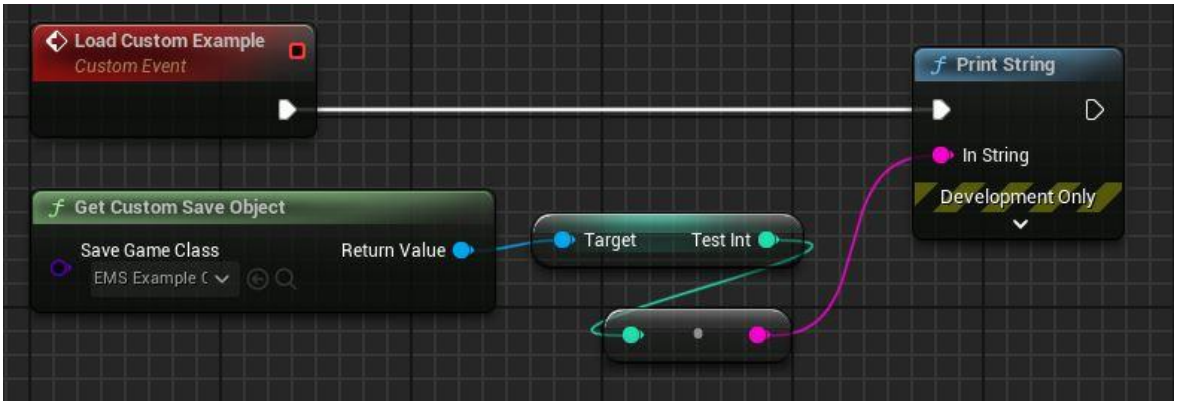
The save node will automatically save all currently cached Custom Save Objects.
An object is cached automatically, once accessed with Get Custom Save Object.
For slot-bound objects, only objects belonging to the current slot are included.



Optionally, you may only save one specific object.



Custom saves don't have to be manually loaded, you can access their data like this.



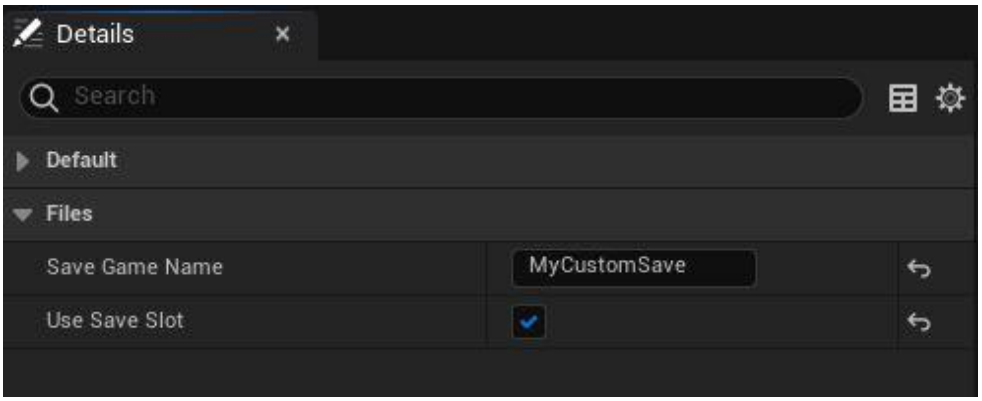
Input Class

The Get function has an auto-cast feature that will return the input class.
Typically, this is your own Blueprint class derived from EmsCustomSaveGame.
You can access the variables directly in all of your Blueprints without limitations.



Class Settings

The name of the custom save file can be defined in the class defaults of your EmsCustomSaveGame Blueprint. It also has an option to be used with the slot system.
By default, the custom save data is unbound and the same for all save slots.
Each Custom Save Object class also requires a unique name, unless you override it.

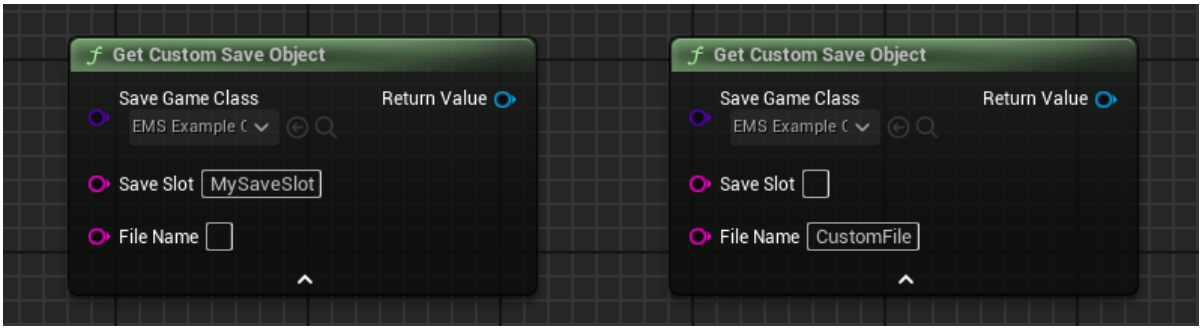


Advanced Use

Optionally, you can access a Custom Save Object from a specific save slot.
'Use Save Slot' must be enabled for the class. If empty, it will use the current save slot.

You can also define a custom file name. This will override the one from the Blueprint and allows having multiple files with just one Custom Save Object class.

Save Custom Objects requires the 'Save Game' input from a node using the custom slot.
It will only automatically save cached slot-bound objects belonging to the current slot.



Other Notes
Custom Save Objects are always valid and a validation check is not required.
You can use the Does Custom Save Exist node to check if the file is actually available.

You can also conveniently reset the data using the Reset Custom Save Object node.
An example would be to restore default settings.

3.8. Thumbnails

Easy Multi Save also supports saving thumbnail images for save slots. This is done by saving the captured image of a 2D render target source. It can be really useful for a save game UI. The Example Project shows how this can be done. In the EMS_HelperFunctions library, you find a Blueprint function named Save Thumbnail Image, which shows how to set up the render target capture.

The capture source should render as Final Color LDR in RGB .

Make sure the Render Target Texture has the RGBA8 SRGB format. In case the thumbnails are too dark, try adjusting the contrast of the Scene Capture Component Post Processing.

You can choose the compression format in the Plugin Settings. Either Jpeg or Png.
If you experience a small hitch with the high quality Png format, try switching to Jpeg.

Thumbnails are stored as default *.sav file, but can be opened like any other image.

Chapter 4. Extended Use

4.1. Multi Level Saving

The Multi-Level saving feature is for keeping save data between levels.
Example: Save in Level-A, go to Level-B, save in Level-B, return to Level-A, load Level-A and the data is still there.

Multi-Level Modes

In the settings, you can choose between a few options for how Multi-Level saving is handled. With all available methods, Game Mode and Game State are always persistent and not saved per level. It will also automatically save the Player transform per level and keep all of its data around without the requirement to do anything manually.

Disabled
Multi-Level saving is disabled.

Basic
This method is suited for projects that use separate levels without streaming.
It will keep data between multiple persistent levels.
Practical Use: Linear Game with multiple Persistent Levels. Shooter.

Streaming
This method is for one Persistent level with streaming or World Partition Sub-Levels.
Practical Use: Open World Game with one Persistent Level. City Builder.

Full
Supports multiple Persistent levels with streaming or World Partition Sub-Levels.
Practical Use: Open World Game with multiple Persistent Levels. Soulslike.

Linear Games
If you are building a linear game with consecutive persistent levels, where data from previous levels is no longer relevant, you can call Clear Multi Level Save before saving. This removes obsolete level data while preserving Player and Game Mode data, allowing them to carry over to the next level.

Compatibility
Compatibility for save game files with different Multi-Level saving modes is restricted. Basic is compatible with Full ; Disabled is compatible with Streaming.

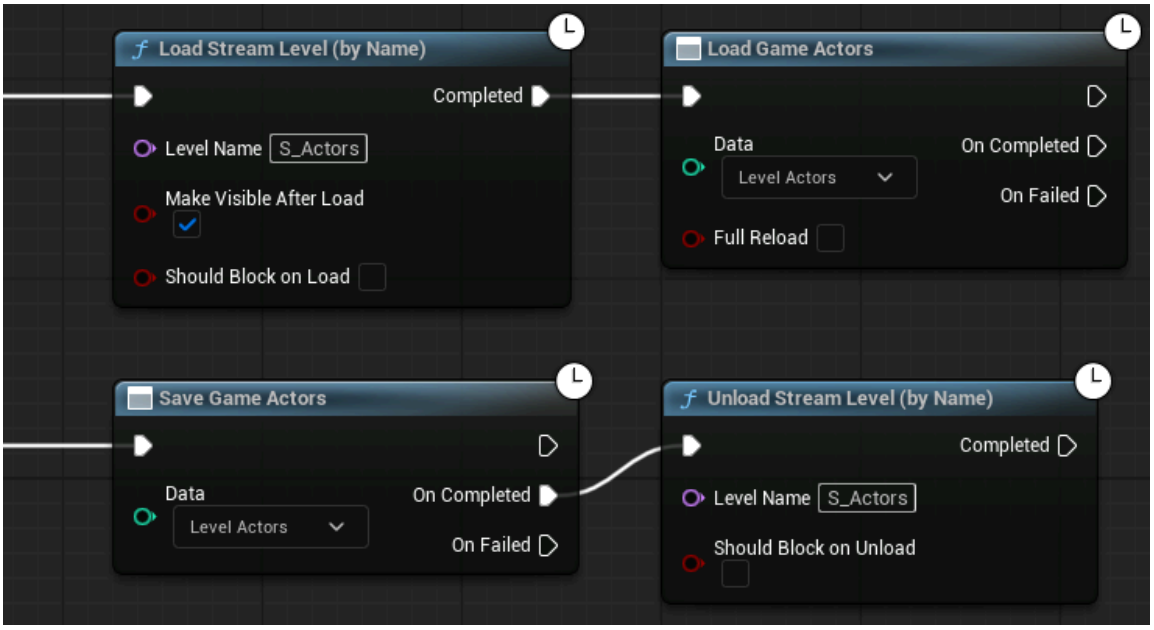
4.2. Level Streaming

Saving and loading Streaming Levels works with the default Unreal functions and no additional setup is required. Multi-Level Saving must be set to Streaming or Full.

Automatic Streaming Save
The simplest approach is to enable the plugin setting for Streaming Saving.
It keeps the state in memory and requires a call to Save Game Actors to write the actual data to disk.
An initial call to Load Game Actors must be issued before automatic streaming save/load begins.

Manual Streaming Save
When manually saving Actors in a Sub-Level, the level must be loaded and visible when Save Game Actors is called. The data is written directly to disk. Use this manual workflow when Streaming Saving is disabled or when your project intentionally controls restoration itself. When automatic is enabled, an additional manual load is unnecessary.

To restore Actors from streamed Sub-Levels, call Load Game Actors once the streaming process has completed. Ensure Full Reload is disabled. Otherwise, all Actors will be reloaded instead of only those in the streamed level.



Level Instances
Requires Allow Dynamic Streaming Levels to be enabled in the Plugin Settings.

It enables saving Runtime Level Instances, multiple Streaming Levels of the same type, and Actors inside Level Instances within Streaming Levels. World Partition levels are unaffected. They handle Level Instances automatically. The setting is not backwards-compatible, as it generates different unique IDs for Actors.

Due to an engine limitation, *Streaming completion is unpredictable* with Actors in Level Instances within Sub-Levels or dynamically spawned Level Instances. The Stream nodes often complete too early before streaming has finished. Persistent and World Partition levels are more reliable.

For dynamically spawned Level Instances:

→ Set the *Optional Level Name Override* parameter so each instance has a stable unique ID.
→ After spawning, add a Delay before calling Load Game Actors.
Or use the Is Level Streaming Active node to wait until streaming has finished.

4.3. World Partition

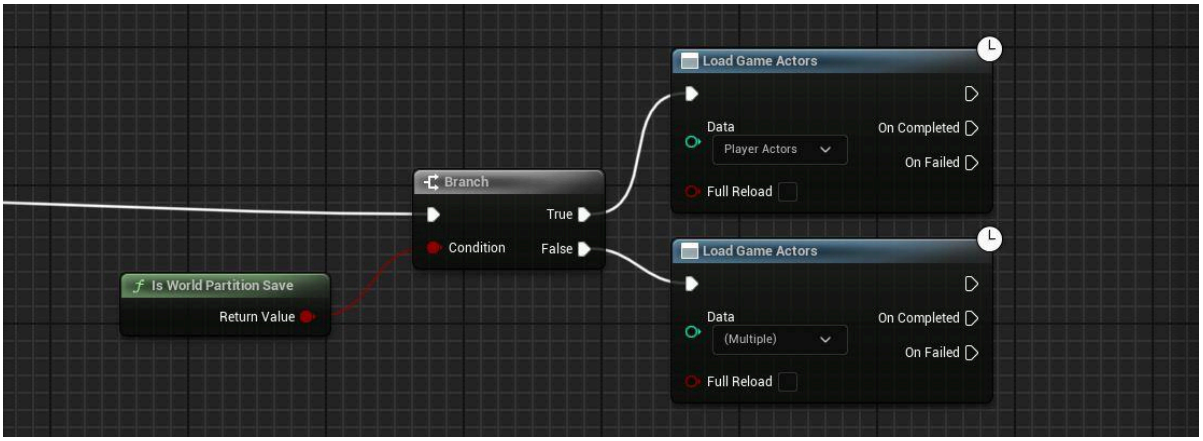
World Partition and Level Streaming use a unified pipeline and work in similar ways.

Placed Actors in World Partition regions are automatically saved and loaded, as long as Multi-Level Saving is set to Streaming or Full. To further customize how World Partition data is managed, you can adjust several Plugin Settings.

Default Initial Loading

By default, an initial load of placed and runtime Level Actors will happen at level begin. The system waits until the initial cells have streamed in and the streaming-state is stable before proceeding.

Make sure that you don't manually load Level Actors at the start and only keep Player Actors checked with Load Game Actors. You may also use the Is World Partition Save node, if you have other persistent levels without World Partition.



Skip Initial Loading

You can also skip the initial load on begin by changing World Partition Init Mode in the Plugin Settings. In this case, you must manually load Level Actors at begin as with Level Streaming. If manually loaded at least one time, the system will start to auto save/load the cells. In most cases automatic initial load is recommended.

Player Character

There might be a race condition where the World Partition system is not streaming in the required regions fast enough, so the Player may fall through the world. A reliable solution is to lock the Character location and disable its physics and input until there is a valid floor with collision. You can use 'Find Floor' from the Movement Component.
This is demonstrated in the Example Project.

Performance

If you have less than 50 Actors per cell, you can use the Default loading method, as the performance impact is not significant. Otherwise you can use Multi-Thread or Deferred loading.
See Load Methods.

Reloading

If you want to reload a World Partition level, it is recommended to re-open the level and also call Clear World Partition Save before.

Destroyed Actors

The easiest way is to use automatic Actor destruction, which is enabled by default in the Plugin Settings. This way, all destroyed Actors are handled automatically in each cell.

Physical Actors

Saving/Loading placed physical Actors works out of the box in a partitioned world. Spawning runtime physical Actors should be avoided, since those are always saved in

the persistent Level, and it's uncertain if the region they are in, is actually visible and has collision. It is possible to also lock them in place and check if they have a valid floor etc.

4.4. Load Methods

Having huge amounts of Actors in your level can be potentially expensive to load.
For this case, EMS supports different methods to load level Actors.
The Player always uses the default method. You can change load methods in the Plugin Settings .

Default
Simply loads all Actors in the game thread at once.
This works for a small or medium number of Actors and is likely sufficient for most projects.

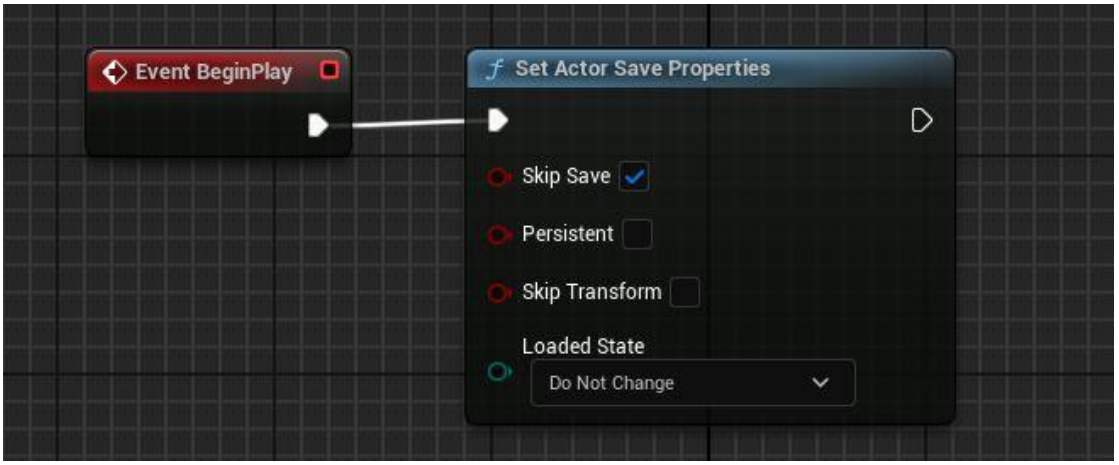
Deferred
Deferred loading distributes the loading across several frames. The system batches up a small amount of Actors, loads them at once and then repeats this as many ticks as it needs to load all actors. Good for large amounts of spawned runtime Actors or Actors with complex data.

Multi-Thread
This method will perform the major operations on a separate thread.
Good for large amounts of level placed Actors. Overall, Deferred load is generally favorable.

4.5. Save Properties

By using the Set Actor Save Properties node, you can change the behavior of how an Actor is saved and loaded. The inputs are self-explanatory for the most part.
Ideally, this node is used from the BeginPlay event of the Actor.

Skip Save
There are some cases where you may want to exclude an Actor from getting saved.
In this case, you just check Skip Save.



Persistent Level Actors
Non-player Level Actors can be set to persistent and restored independently of the current level.
Persistent Actors will always appear at (0,0,0) when spawned.
Make sure to call Load Game Actors before saving again in a new level.
Persistent Actors are not kept in Multi-Level archives as they are the same in each level.

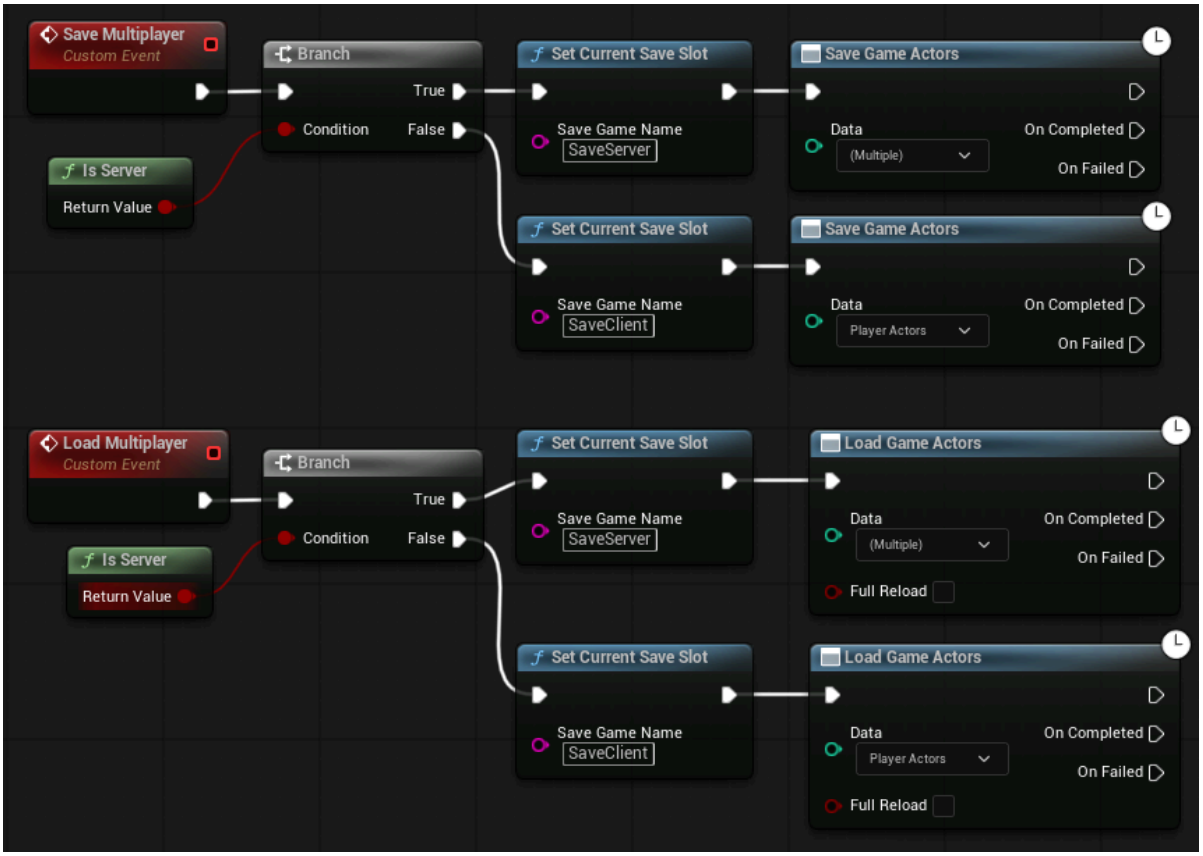
Skip Transform
When Skip Transform is checked, the Actor will not save and load its transform.
Placed Actors retain their current transform.
Runtime Actors are respawned at the default transform and must be repositioned manually.

Loaded State
Change if an Actor is currently seen as being already loaded. It ties into the 'Full Reload' setting of the Load Game Actors node as well as into the Is Loaded State node.

4.6. Multiplayer

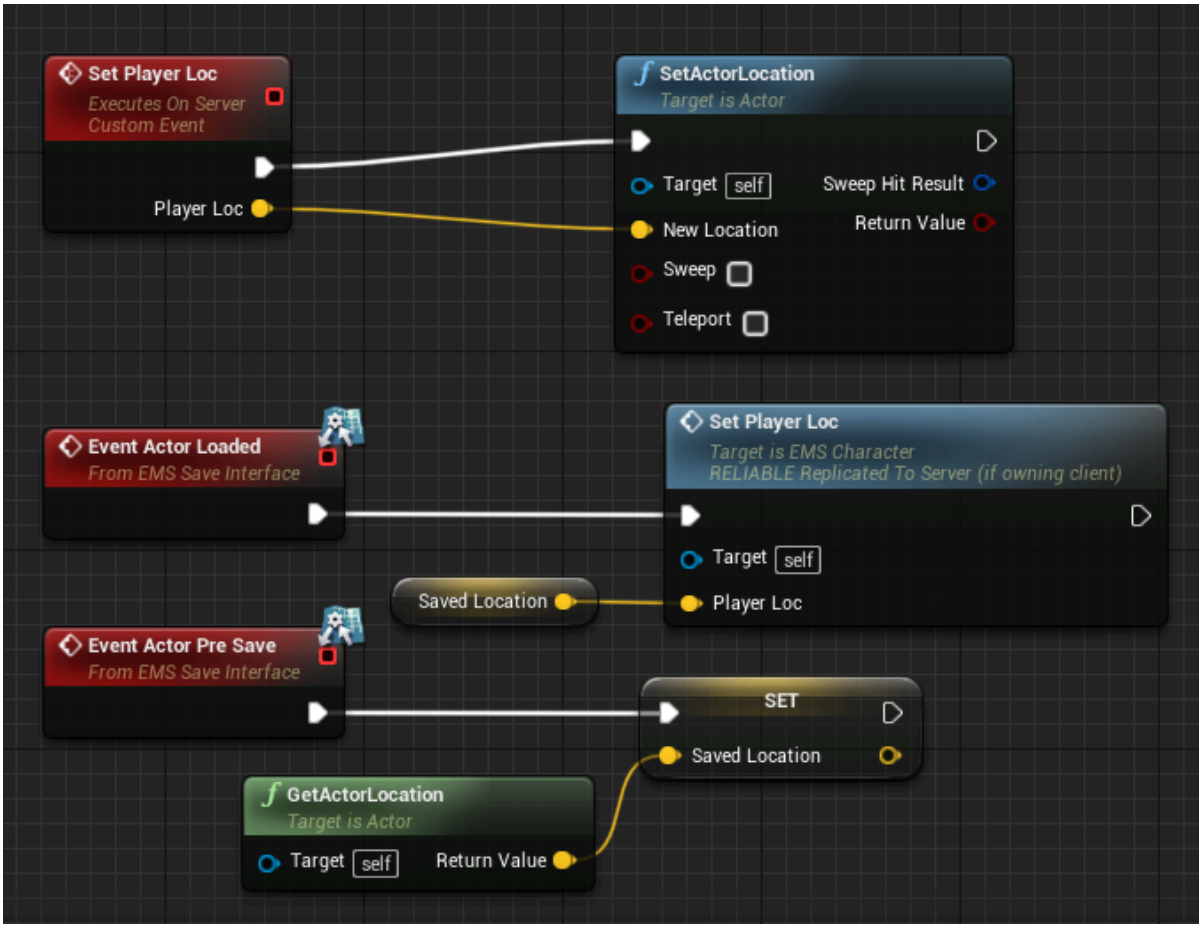
Saving in multiplayer games is a complex topic, there are many pitfalls and considerations. Level Actors will always be handled by the server, which does not differ from saving in single player games.

Peer-to-Peer
In a basic P2P multiplayer setup, you can save the Player Actors on the local client only. This would look something like this:



Client Replication
A client can save the replicated data of its owned Actors. Controller, Pawn and Player State. You will have to pass all saved variables(including transforms) through a replicated function call, so the server knows about it. The automatic saving/loading of transforms has no effect on a client.

Think of loading data on the client as of any other client action. You will have to tell the server about game relevant data and replicate it accordingly.
In practice, it looks something like this:



Listen Server
The Listen Server can save/load its own Player as well as the Level.

Dedicated Server
Level Actors are saved on the server side. As for the player, you can store client data on the server and/or let players save their data locally.

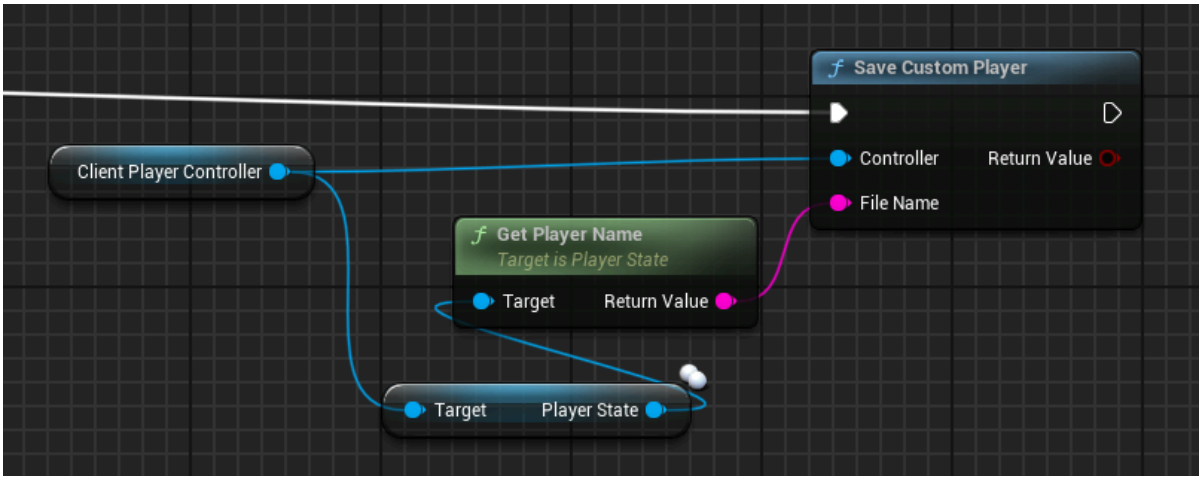
Seamless Travel
This is supported without any further considerations.

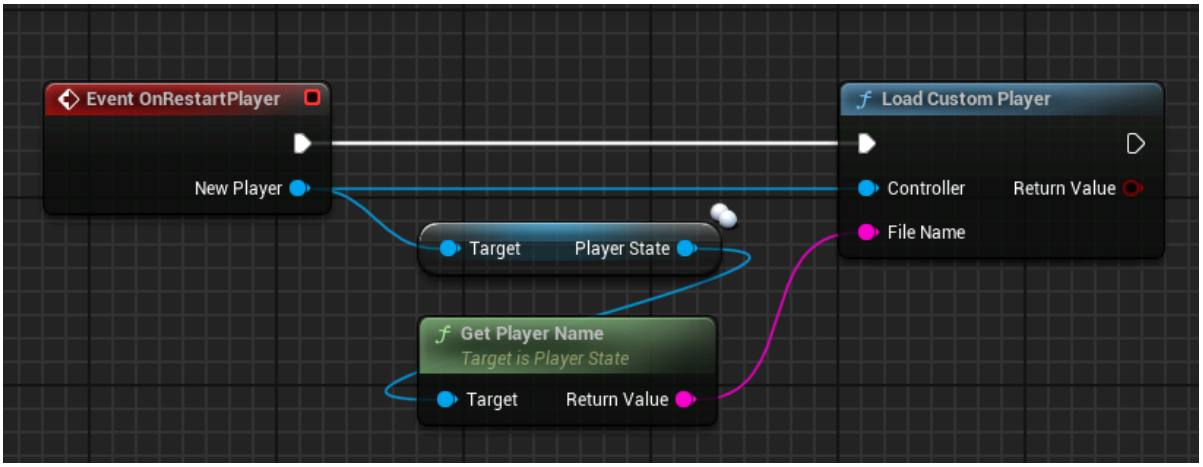
Server Only Player Data
You can easily store individual player data exclusively on the server.
It just requires a unique player identification, such as the Steam ID.
This way it will create separate save files for each player on the server side.

When to actually save the player data is up to the individual project, theoretically the server can just save all players at once, or do it when a player attempts to leave or has logged out.

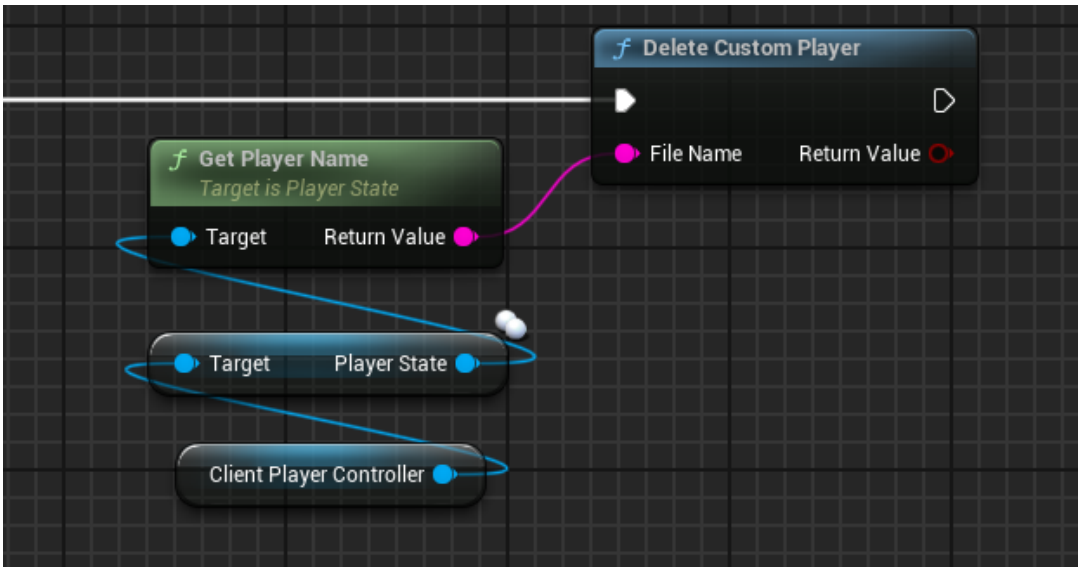
Custom Player functions must execute on the server. A client may request the operation through a validated server RPC, but the server must resolve the target Controller and choose the authoritative file identifier. It's generally recommended to not request it from the client side.

It will always use the current save slot to store the data. It’s a very lightweight function.





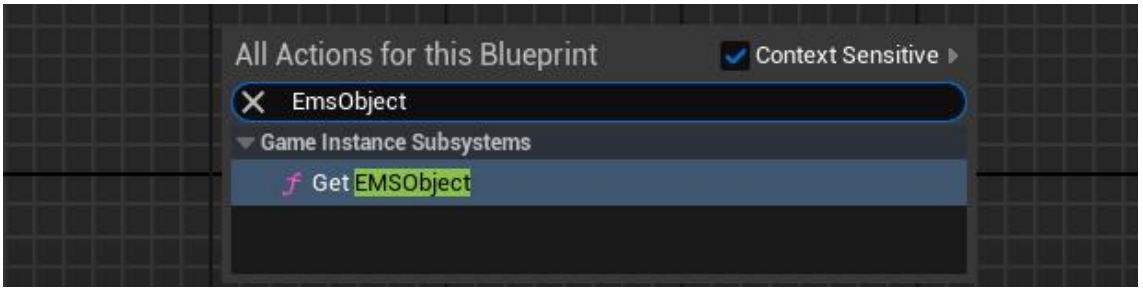
Custom Player Data can also be removed like this.



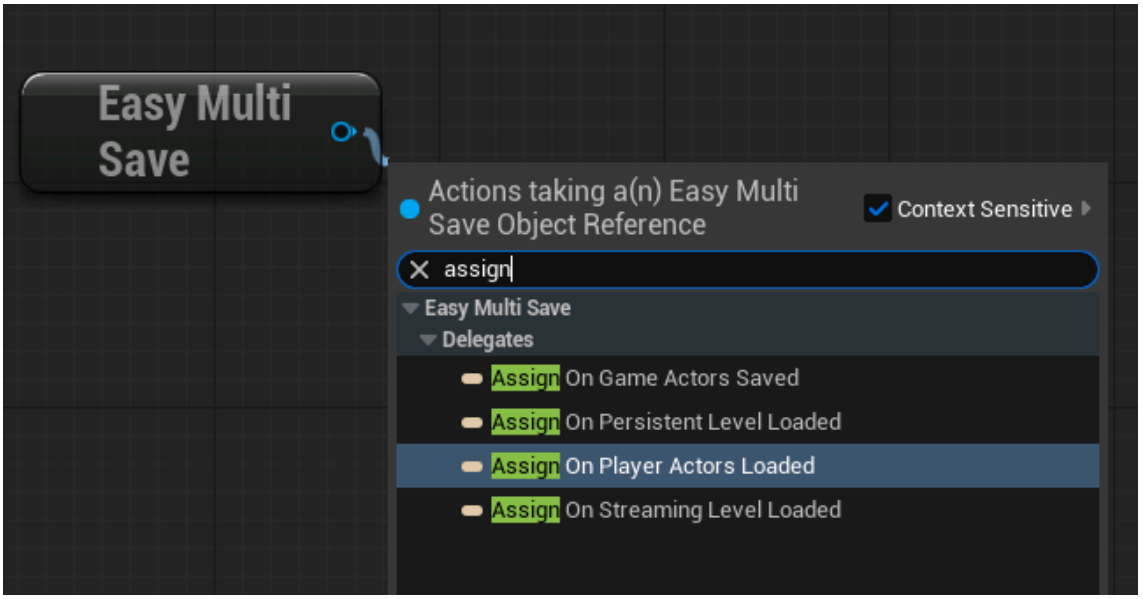
4.7. Blueprint Delegates

The plugin has several delegates that are broadcasted when Actor saving or loading has completed.
Without these, you would usually rely on the 'On Completed' output from the async nodes.

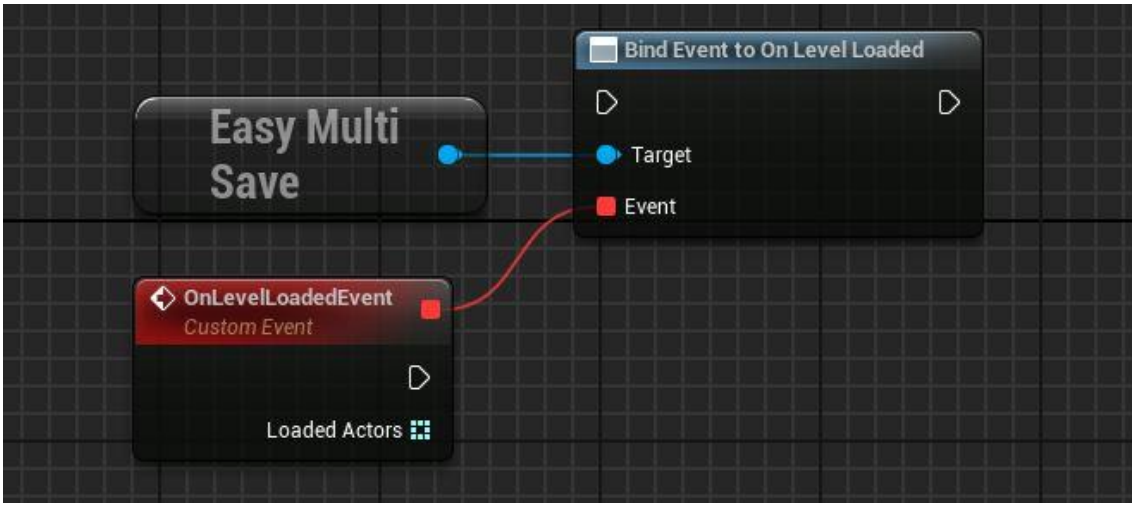
To access these Delegates, you need to look up the Easy Multi Save Subsystem. Just search for EMSObject in the Blueprint Actions list.



From there, you can currently access four Delegates with obvious functionality.



You may bind or unbind them as usual. The Level and Partition events also come with a list of Soft-References to all loaded Actors, also including runtime spawned Actors.
The list is only valid on the frame of the event execution, but you may copy it as you see fit.



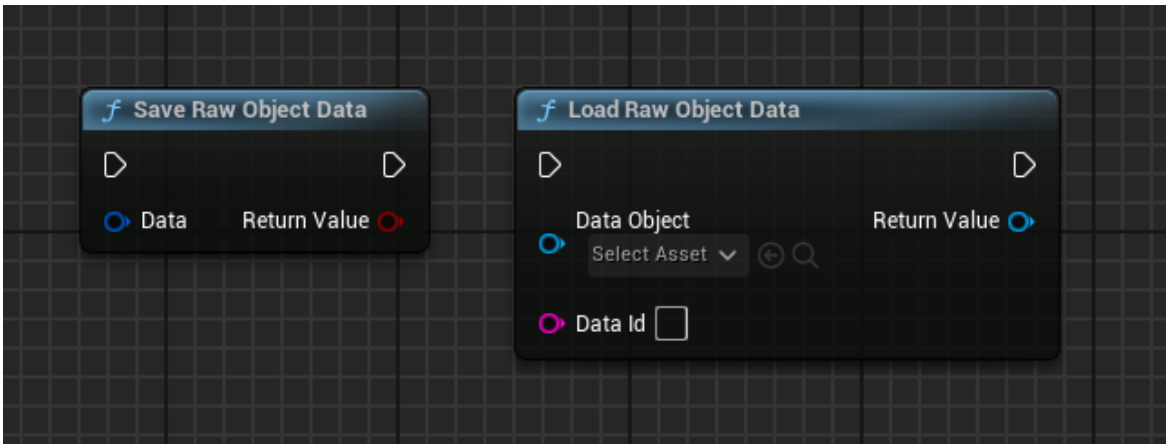
Chapter 5. Advanced Use

5.1. Raw Object Saving

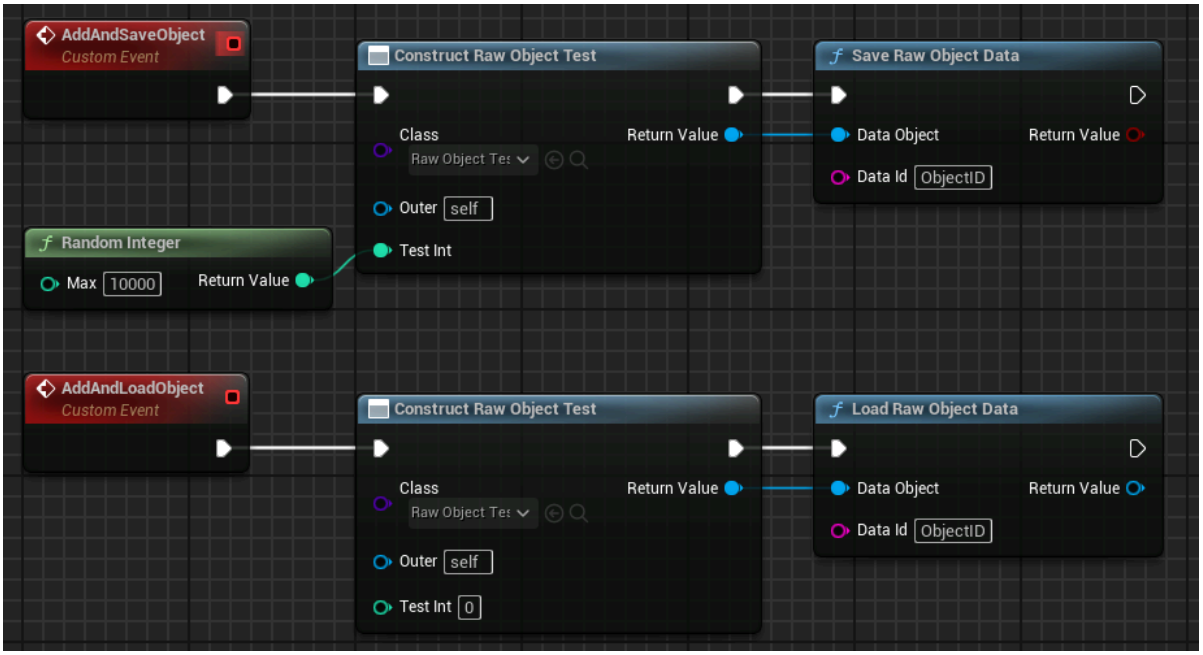
UObjects can be saved as part of any Actor.

The objects can be anything. Runtime created UObjects, Subsystems or other Actors. Variables inside the objects also need Save Game checked to be considered.

The functionality just requires two nodes inside your saved Actor Blueprint. The input Data struct takes the Object you want to add to the Actor save data and also a unique local ID. This can be anything, such as a GUID, Gameplay Tag or a simple number.



The data is then saved alongside the Actor. A good point to call the save node is in the Actor Pre-Save event. When loading the Actor, EMS does not automatically re-create these objects, instead it will retrieve the data on demand. It has no performance cost, as the data was pre-loaded with the Actor.



This can also be used to store data for Player Owned Actors, such as a Weapon that is spawned and equipped on startup by your Character. It is also recommended to not add the Save Interface to the Weapon in this case.

These nodes can only be used inside Actor or Component Blueprints.

5.2. Object Collections

This feature allows you to serialize and store any number of UObjects directly to their own file.
It can be used for the Game Instance, Subsystems, and even individual Actors.

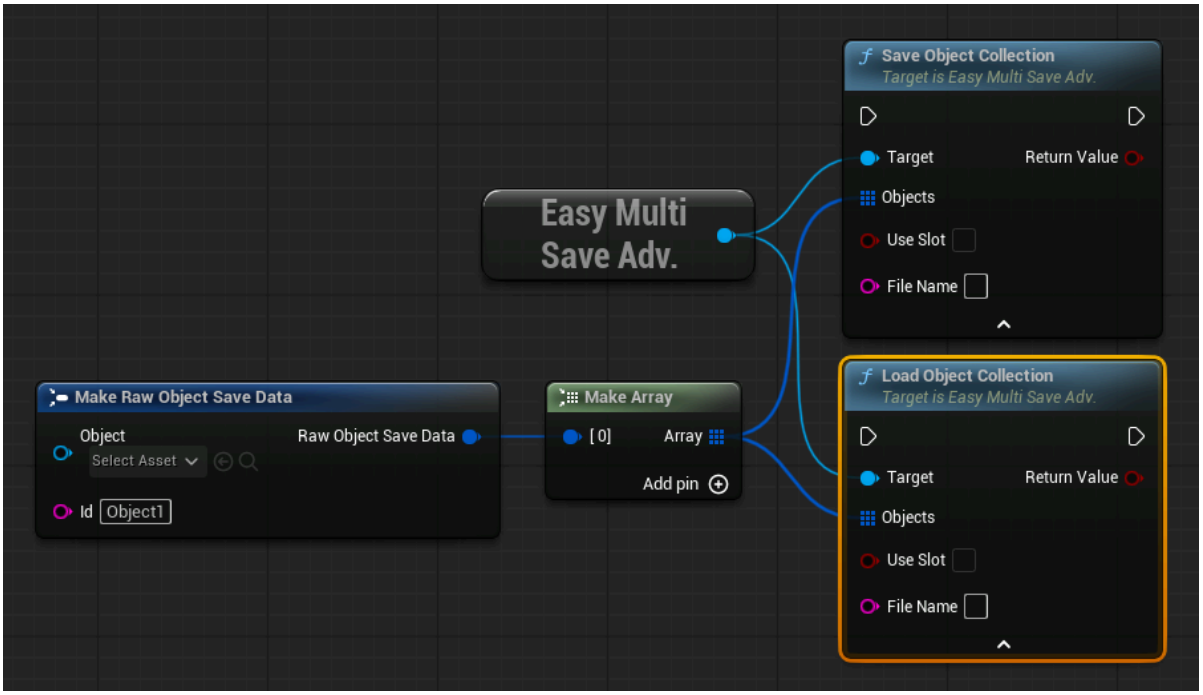
Similar to Blueprint Delegates, you just get the appropriate subsystem reference.
In this case, search for EMSObjectAdv in the Blueprint Actions list.

Compared to Raw Object Saving, Object Collections are fully decoupled from the standard save framework, but still operate with Raw Object Data as input.

Files may be saved to the current Save Slot as well. You can optionally define a custom file name(default ObjectCollection.sav), and you are free to distribute objects across multiple files.
There are no inherent limits on the number of objects or files used.

Variables inside the Objects also need the Save Game flag to be considered.

Loading works similarly to saving, using identical function inputs to load the stored objects from disk.



The Example Project features a simple example in the EMS_FunctionOverview Blueprint that shows how to save the Game Instance using Object Collections.

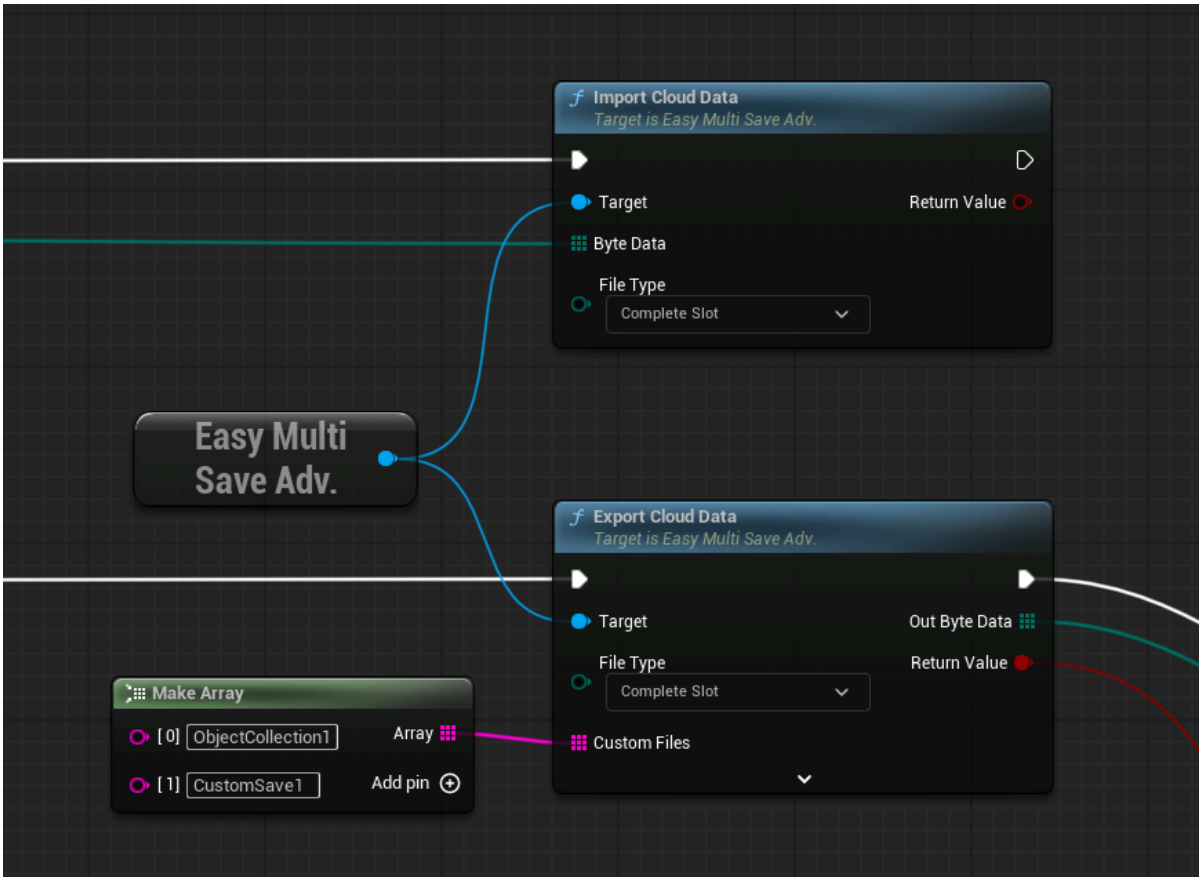
5.3. Cloud Integration

While Steam makes it very easy to upload cloud saves via complete files, other backend systems, such as Epic Online Services (EOS) require you to provide the save as raw byte data for upload.

Using this method is also recommended for Cross-Platform saves.

You can seamlessly export and import raw byte data and sync the complete Save Slot and/or Custom Save Objects and Object Collections. This data can then be fed directly into the native cloud saving nodes, for example from EOS.

This is considered an advanced feature that you also find in EMSObjectAdv.
Import Cloud Data and Export Cloud Data are everything you need. It uses files from the current Save Slot by default and resolves all paths automatically on import. After importing data from the cloud, you can call the load functions as usual.



Network drops and sync errors are the biggest dangers of cloud saving. To ensure player saves are never overwritten by corrupted cloud data, the Import node features mandatory security and integrity checks before any data is written to disk. You may also just set a different Slot on import if you don't want to overwrite data in the current Slot.

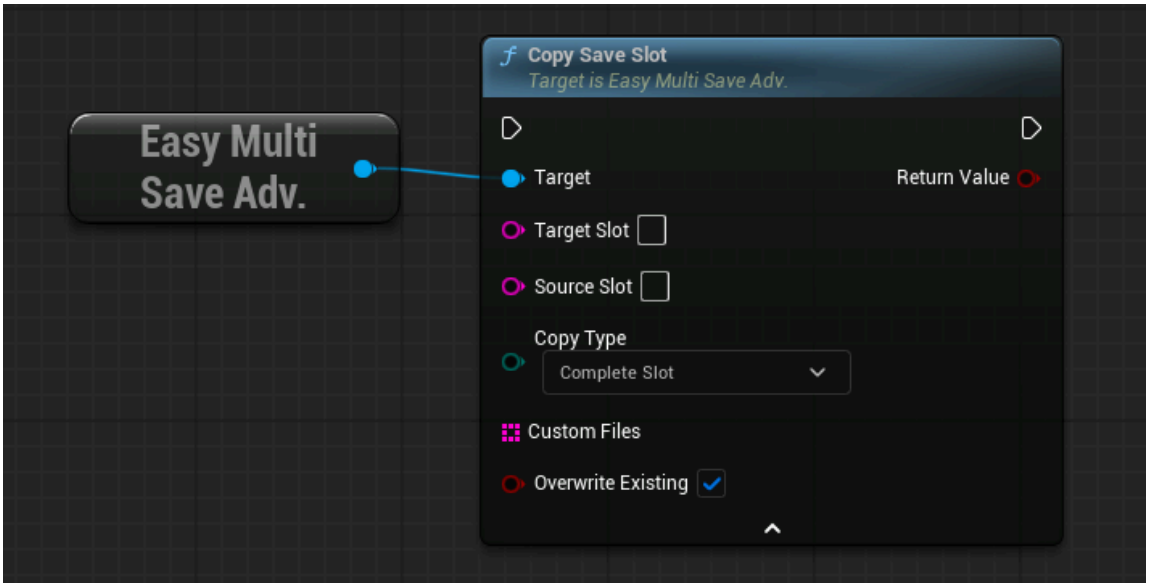
5.4. File Operations

File operations can copy or transfer data between slots.
This is considered an advanced feature that you also find in EMSObjectAdv.

Copy Save Slot
Copies the persisted data of one save slot to another. By default, the Source Slot is the current save slot, while Target Slot defines where the copied data is written.

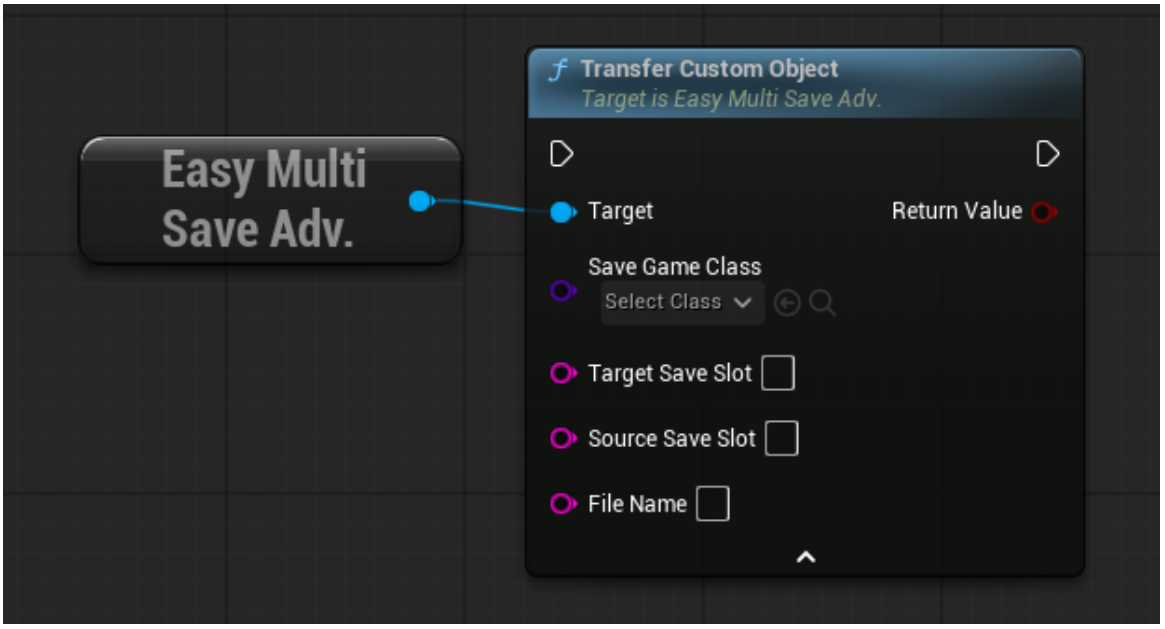
The operation can copy the regular slot files as well as explicit slot-bound Custom Save Objects. Existing target data can be replaced depending on the node settings. EMS validates the source data before modifying the target and updates the target Slot Info accordingly.

Works on data already written to disk and does not copy unsaved runtime state.



Transfer Custom Object
Transfers a slot-bound Custom Save Object from one save slot to another. The object is written to the Target Slot without changing the current EMS save slot or modifying the source save file.

This is useful when selected Custom Save data should be carried over between slots without copying the complete save slot. The target Custom Save file is replaced with the transferred data and the target Slot Info is updated accordingly.



5.5. Data Redirectors

In case you need to update existing save data with renamed Levels or Actor classes, you can do so in the Plugin Settings. This will also correctly update all saved Actor references.



↓ Runtime Actor Redirects
Renaming an Actor instance or re-parenting a Blueprint does not break save data. However, replacing a runtime-spawned Blueprint class changes its class path and can cause issues. Runtime Actor Redirects allow you to map old Blueprint class paths to new ones, so existing save data continues to spawn the correct class.

Setting
Key: Old Blueprint class path.
Value: New Blueprint class path.

How to get the class path
Right-click the Blueprint in the Content Browser.
Select *Copy Object Path*.
Paste the path (e.g. /Game/Path/MyActor.MyActor).
Append *_C* to the asset name (e.g. /Game/Path/MyActor.MyActor_C).

↓ Level Redirects
Level Redirects map old level package paths to new ones to keep existing save files functional after levels are renamed or moved. This also works for streaming levels, but currently not for level instances that contain saved Actors.

If you rename a level multiple times, just keep the original Level as Key. Avoid replacing the old level asset with a new level of the same name.

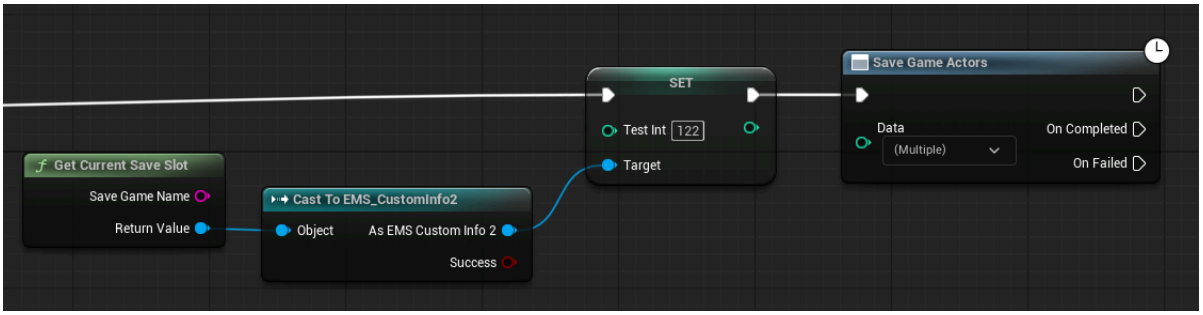
Setting
Key: Original level package path.
Value: New (canonical) level package path.

How to get the level path
Right-click the level in the Content Browser.
Select *Copy Package Path*.
Paste the full path (e.g. /Game/Maps/MyLevel).

5.6. Custom Slot Info

Using a custom Save Slot Info is easy as well. You need to create a child Blueprint of EMSInfoSaveGame and then cast to it from Get Current Save Slot.

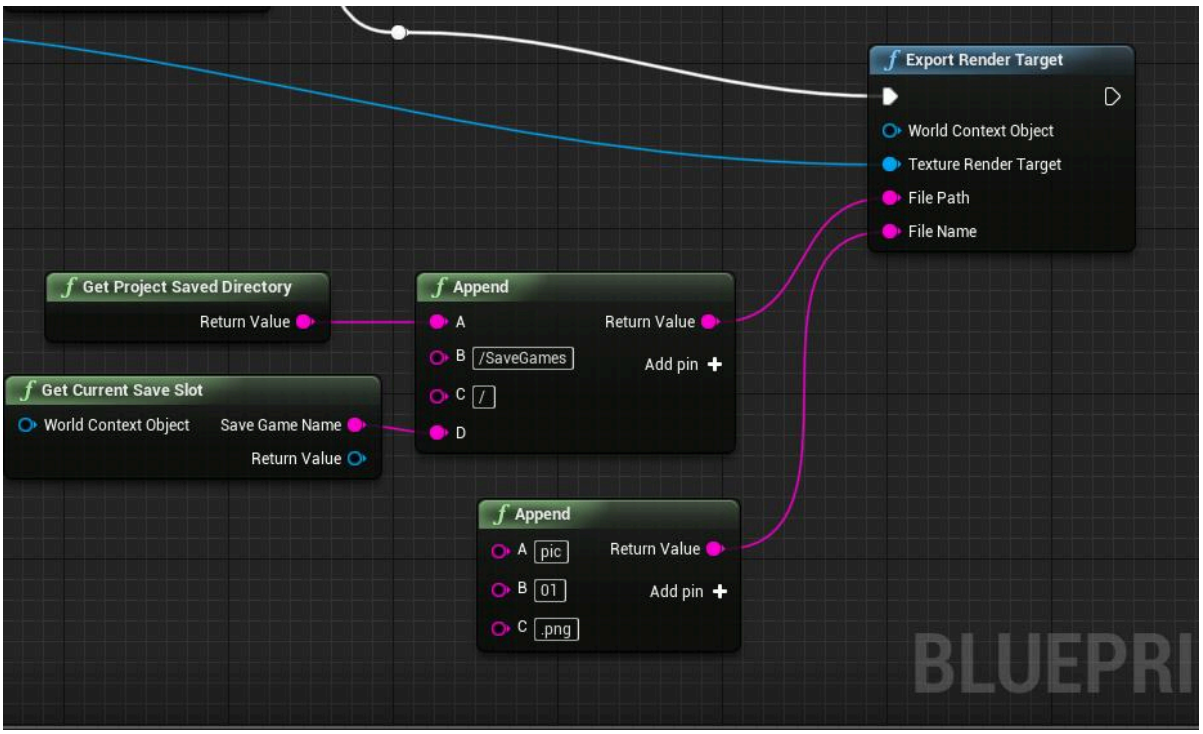
Since the Slot Info is always saved automatically, you need to make sure that you set the variables before calling Save Game Actors.



5.7. Additional Data

Sometimes you want to save additional data as part of a Save Game Slot.
For example, a Render Target from a panorama photo mode in your game.
The only thing you need is to set up the correct paths.

In this example, a Render Target is exported to the Current Save Slot and saved in the appropriate folder.



5.8. EMS Addons

This is a **free** plugin providing reusable helper extensions for Easy Multi Save.
It is available to all verified users on Discord.

Description
While Easy Multi Save provides the core save and load framework, certain systems require additional Blueprint logic or project-specific handling. Easy Multi Save Addons

extend the core system with ready-made modules to speed up your workflow even further.

Most of the functionality provided by EMSAddons could also be implemented directly in Blueprint.
The goal is to reduce that setup while keeping the solutions streamlined, reusable, and easy to maintain.

These extensions intentionally remain separate from Easy Multi Save itself.
They target workflows that are too narrowly focused for the core plugin, allowing EMS to stay lean and broadly applicable.

Included Modules

Autosave Manager
Convenient helpers for timed, triggered, and checkpoint-based autosaves.

Actor Spawner
A reusable runtime actor spawning workflow similar to the Actor Spawner included with the EMS example project.

World Partition Manager
Allows Level Actors to freely travel between World Partition cells while correctly following streaming.

EMS Geometry Collection
Automatically saves and restores fractured Chaos Geometry Collections and their individual pieces.

EMS Level Sequence
Automatically saves and restores relevant Level Sequence playback state and properties.

Instance Manager
Handles runtime changes to HISM/ISM instances and painted Static Mesh Foliage.

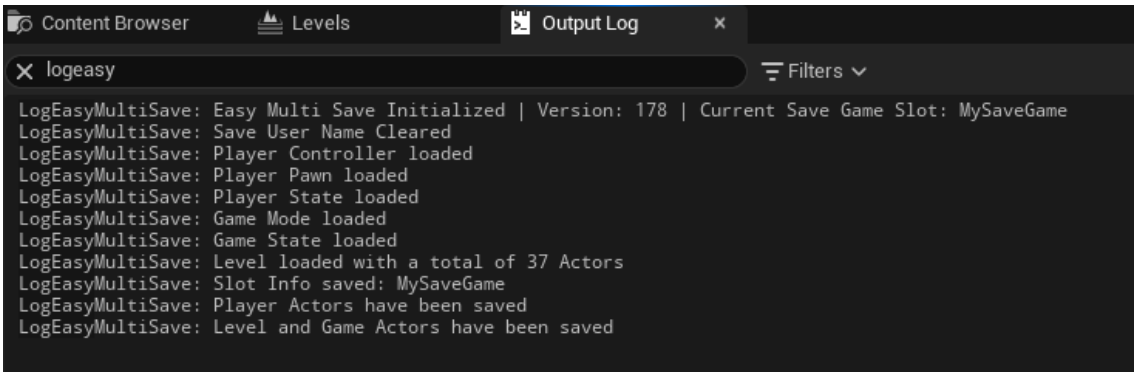
Physics Guard
Prevents physics objects from falling through the world when the supporting streamed geometry unloads.

Chapter 6. Framework

6.1. Debugging and Testing

Logging
The Output Log will give you a good overview of what EMS currently does.

Type "Logeasy" in the log filters, to quickly see only messages from EMS.

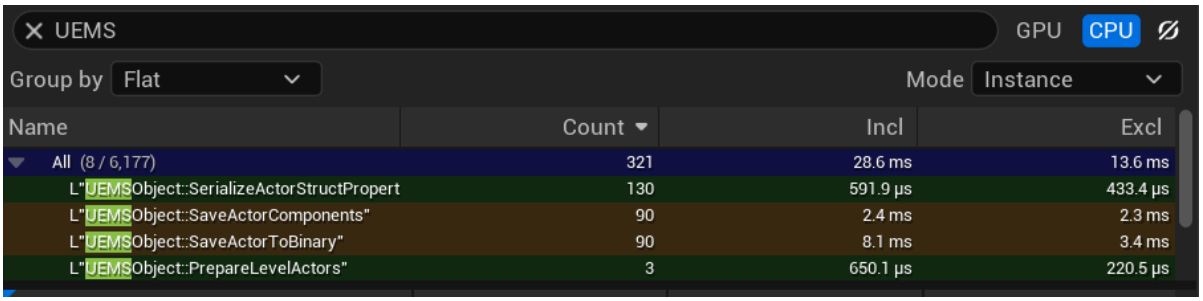


Verbose Logging
To get more information from the Log, you can use verbose logging.
Add the following to your DefaultEngine.ini:

[Core.Log]
LogEasyMultiSave=Verbose

Or for very verbose logging, which logs every saved/loaded Actor:
LogEasyMultiSave=VeryVerbose

Unreal Insights Profiling
The plugin has some advanced information for the Unreal Insights Profiler. You can simply search for UEMS to get all CPU scopes and identify any performance bottlenecks that you might face when processing huge amounts of data and/or save complex levels.



Data Size Logging
To inspect how much data is stored per saved Object, enable data size logging in the config.
This will print the serialized size of each Object/Actor to the Output Log during save operations.
Simply enable Log Object Data Size in the Plugin Settings .

Advanced Save Report
This will write a complete mirror of the binary save files in fully readable *.txt format into *Saved/Logs/Ems*.
In addition, you will get a lot of information about Actors that is not part of save files.
Simply enable Advanced Save Report in the Plugin Settings and set it to *OnSave* or *OnLoad*.
You can open the text files directly in your browser with this Easy Multi Save File Inspector.

Standalone Testing
Using standalone testing from the editor can lead to unexpected behavior. If you want to make a proper standalone test, it is recommended to package the game as development or shipping.

6.2. Project Based Plugin

In some cases, you will have to package Easy Multi Save alongside the project.
This is always the case if you want to modify the plugin source code. It is also required when building for certain mobile devices or consoles.

Create a **Plugins** folder inside your Project directory. **\...\MyProject\Plugins**

Go to the Folder where UE is installed and pick the version your project is using.
\...\Epic Games\UNREAL_VERSION\Engine\Plugins\Marketplace

Copy the **EasyMultiSave** folder to the newly created **Plugins** folder inside your project directory. In the EasyMultiSave.uplugin file set the 'Installed' entry to false.

Now open the Epic Launcher and uninstall the Easy Multi Save plugin, so you only get the one from the project. Please note that you have to install EMS again from the launcher, if you want to use it with other projects.

For the plugin to compile and package, the project needs at least one C++ class. Otherwise it will always use the version from the launcher.

6.3. Consoles

Build
First, you need to change the file system to Console in the settings. Console Versions of the plugin have to be built from the project folder. See Project Based Plugin.

You also need to add the appropriate Consoles to the EasyMultiSave.uplugin file.
For example:

```
"PlatformAllowList":
[
"Win64",
"PS5",
"XB1",
"XSX"
]
```

Also set *bAllowConfidentialPlatformDefines = true;* in the EasyMultiSave.build.cs file.

File Access

The node Get All Save Slots may be limited or not available on certain platforms, due to file access restrictions. A workaround is to use a set number of static save slots that are sorted by the timestamp provided from Get Named Save Slot. A Slot in EMS is just represented by a simple string, so this will not cause any trouble.

Save Users
This is handled by the console login system.
The Save User functions from EMS do not work on consoles.

Save Slots
On PlayStation, avoid slot names containing 'x' since the system may interpret it as a concatenation character rather than a label.

6.4. Android

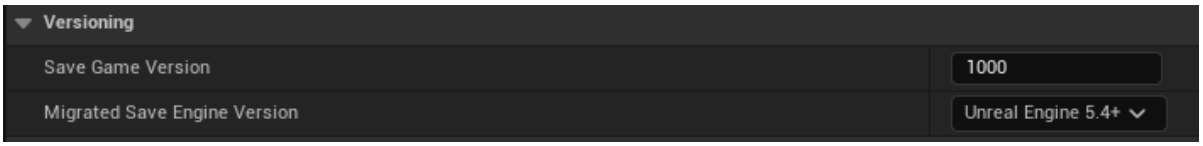
If you have issues with permissions on Android, check out these resources:
https://www.fab.com/listings/009c8a8a-8260-4b40-8873-153cb70f5f80
https://forums.unrealengine.com/t/android-13-save-game-permissions-not-working/

You can call the EMS functions as usual. You don't need to call them from the Game Instance.

Modern Android versions use app-specific save directories that normally require no storage permission. **android.permission.WRITE_EXTERNAL_STORAGE** is relevant to legacy Android targets and does not grant additional access when targeting Android 11 or later. Verify the actual save location and target SDK before changing permissions.

6.5. Versioning and Integrity

Internal Versioning
All files from EMS are automatically versioned. By default you won't have to do anything. Unless migrating save files from engine builds previous to Unreal Engine 5.4. In the settings, make sure that you select the engine that the old save files were created with. It can go back to UE 4.27.

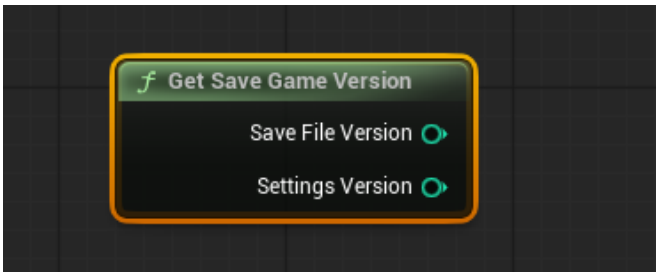


For very old save files, the log will show a warning, informing about a legacy file version. If everything loads correctly, you can save again and new version info is written to the files.

Game Version
In the Plugin Settings, you can define a Save Game Version for your save files.

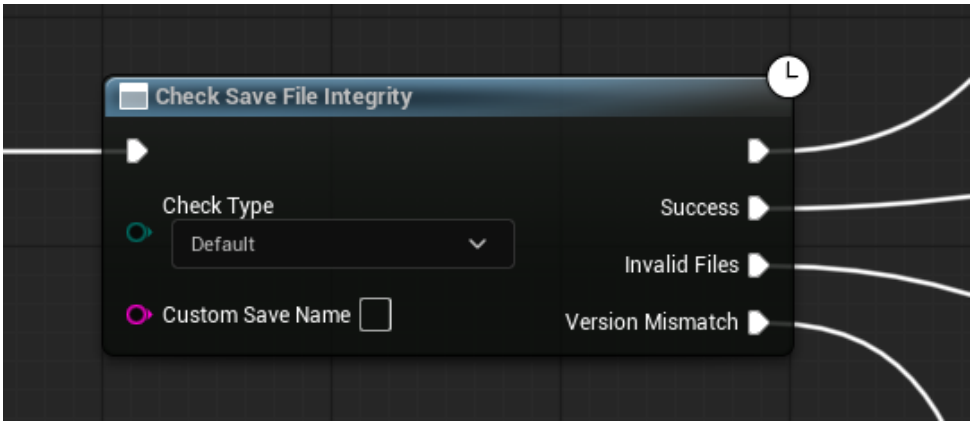
Use the Get Save Game Version node to retrieve either the version stored in the *latest accessed* save files or the currently configured version.

Version mismatches will not automatically cause loading to fail. The Game Version is intended as a soft compatibility check rather than a strict validation mechanism.



Manual Integrity Check
The Check Save File Integrity node verifies save files and ensures they can be loaded without issues. While file corruption is extremely rare(usually because of hardware failures), running an integrity check can still be useful for additional validation and peace of mind.

By default, the node validates Slot, Player, and Level save files. These can also be checked individually, and you may additionally validate Custom Save or Custom Player files as needed.



The integrity check also validates against the Save Game Version defined in the Plugin Settings and provides a separate output pin for version mismatches.

6.6. Automatic Backup

Valid save data can be automatically backed up by the plugin.

Simply enable Auto Backup Previous Data in the settings. When enabled, automatically creates a backup of the current Save Data before overwriting it.

In the rare case where your main save is corrupted(hardware errors etc.), backup data from the previous save will be recovered automatically on load.

Use Get Backup Name to retrieve the platform-correct backup name. Desktop layouts normally append the backup tag, while Console layouts prefix it. Do not construct the backup name manually.

When deleting a save slot using Delete Save Slot, backups are not removed automatically. To delete backup data, retrieve its name through Get Backup Name, then pass that name to any of the delete functions.

6.7. Asset Compatibility

Many FAB Assets come with their own custom save systems, based on the UE default functionality. In most cases, EMS can be integrated with the built-in systems without the need to reverse engineer the asset.

The easiest way is to use Custom Save Objects and re-parent the available save Blueprint to EMSCustomSaveGame. Then just replace the save nodes and references with Save Custom Objects and Get Custom Save Object. This will provide a good headstart and from there, you can also integrate proper Actor saving.

Easy Multi Save is compatible with Menu System Pro from Moonville Entertainment.

6.8. Troubleshooting

Epic Launcher
If you encounter any installation issues through the Epic Games Launcher, please contact FAB Support [support.fab.com] directly, as Epic is in full control over this process. Plugin developers don't have any influence on the launcher builds.

Editor Startup
If there are issues with initializing EMS or if you encounter broken Blueprint nodes, you have to change the loading phase in the EasyMultiSave.uplugin file.
Change "LoadingPhase": "Default" To "PreDefault" or "PreLoadingScreen"

Output Log
If something does not work as expected, check the Output Log first.
EMS reports very detailed information about its current state.

Packaged Game
If there are issues with a packaged game, this usually happens because the DefaultEngine.ini is out of date and contains an invalid Slot Info. The fix is very easy. Go to the Plugin Settings and set the Slot Info Class to None and then set it to EMSInfoSaveGame or your custom class again.



Make sure the entry inside DefaultEngine.ini looks like this:

[/Script/EasyMultiSave.EMSPluginSettings]
SlotInfoSaveGameClass=/Script/EasyMultiSave.EMSInfoSaveGame

Multi-Thread Saving
On some occasions, it may happen that the Render or Game Thread crashes when Multi-Thread saving is enabled. This happens when accessing Objects directly from the Pre-Save event. To fix this, enable Pre-Save On Game Thread in the Plugin Settings.

Another pitfall are multiplayer games that use saved Actors with 'Net Dormancy', since updating replicated variables on dormant Actors is not thread-safe. In this case it is not recommended to use Multi-Thread saving.

Random Actor not loading
In very rare cases, it can happen that EMS cannot process some Actors right after a level has been opened. To fix this, just add a short delay before calling Load Game Actors.

Save then Load but nothing happens
When you call Save Game Actors and Load Game Actors directly afterward, without reopening the level, make sure 'Full Reload' is enabled, so it can load as new data.

Transform is Zero
Make sure that the Mobility of the Actor is set to Movable for level-placed Actors.
For physical Actors, make sure that the Root Component is the one being simulated.

Saved Struct not loading
If struct variables aren't loading their saved values, check the Save Game flag for those variables within the actual struct asset. Another culprit are Hard-Object References inside Blueprint Structs, in some cases, the Engine can just break the complete struct if a reference is resolved as invalid. Just use a Soft-Object Reference instead.

Game Instance Init
When calling EMS functions from the Init Event in your Game Instance Blueprint, make sure to add a short delay before. Since the EMS subsystem is initialized on the same frame, it can lead to unexpected errors otherwise.

Actor Tags
Do not clear all Actor tags on your saved Actors. EMS uses these to identify certain states.
If you add tags to your Actor, make sure to only manually remove them.

File Access Lock
Sometimes, your project is synced with a cloud service and files are uploaded in real time while developing. Applications such as Dropbox or Google Drive lock the files while syncing with the service. This usually leads to write protection, so EMS cannot overwrite the save files while they upload. Typically, this is a matter of seconds where the files cannot be written and the log issues a warning. The solution is to not sync the project while saving, or ignoring the SaveGames folder in the cloud service app.

Object Fail Warning

In very rare cases, you may get a Failed to find Object.None.None warning in development builds. This can be ignored, since it does not have any effect on the functionality and will not appear in a shipping build. It can be related to a Component or Actor that gets processed by EMS. In some cases, duplicating the Blueprint Class or re-creating the Asset will help. Also try adding a short delay before Load Game Actors.

New Game
When implementing a 'New Game' functionality, it is crucial to clear the data in memory. This ensures that no obsolete or unintended data is loaded, providing a fresh start for the game. It is also recommended to use a temporary save slot name, that is never actually saved to.



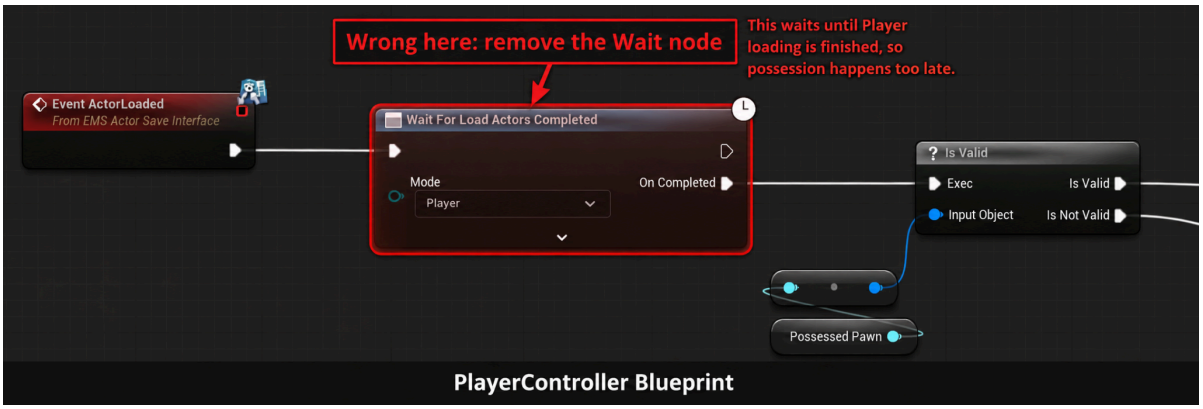
Packaging Errors
Make sure that the plugin only exists in either the Engine/Plugins/Marketplace folder or in your MyProject/Plugins folder. Having Ems in both, can lead to packaging issues.

Save Slot not being deleted
This can happen when the thumbnail is unable to be removed.
The fix is to disable *Always Persist Rendering State* in the Scene Capture Component. Otherwise, the thumbnail file may remain locked and prevent the entire slot from being removed.

File Name Case Sensitivity
Since several non-Windows platforms use case-sensitive file systems, slot and file names are also case sensitive. This means it is theoretically possible to create both *myslot* and *MySlot* as separate files. To avoid internal mix-ups, it is recommended to always force slot and/or custom file names to lowercase on these platforms. EMS does not enforce this rule automatically in order to remain flexible for developers, but it is important to be aware of this behavior.
For UI, a good approach is to display names in PascalCase while saving/loading in lowercase.

Custom Save not being deleted with Slot
This issue occurs exclusively with the console file system when the Custom Save Object hasn't been accessed beforehand, making it unavailable in the cache. As a result, you'll need to manually delete it using the Delete Custom Save Object node.

Possessed Vehicle or Pawn is not loaded
When possessing a saved Pawn/Vehicle from the PlayerController as in the example project,
don't use the Wait For Load Actors Completed node from the Actor Loaded Event. This waits until the Pawn is loaded as well. So the possession happens post load and the old pawn is loaded.



Duplicate Pawns on Possession
When possessing a different Pawn(Like a vehicle) and saving the possession state, it is important that the initial default Pawn and the possessed Pawn are both placed in the level. When initially spawned from a Player Start, it may result in duplicate Pawns being created. You can also save multiple Pawns to different files using Custom Player Saving while they are possessed. You may also use Raw Object Saving and Object Collections for special cases.

Multiplayer Testing
When testing with multiple clients in the Editor, ensure that 'Run Under One Process' is disabled in the Play-In-Editor (PIE) settings. It is also recommended to use separate Save Slots or unique Custom Player Names for each client. Otherwise, they may overwrite each other's saves.

Unreal Engine 5.5 Config Files
If you are using Unreal Engine 5.5, you must always call the relevant Set functions on startup to ensure the current Save Slot and Save User are correctly written to the configuration file. This is a known bug in 5.5 and has been fixed by Epic in UE 5.6.

Appendix

Support

If you have questions, comments, bug-reports, feature requests or want to show a project that uses EMS, feel free to contact me any time.

It would also be very nice, if you could rate EMS on FAB. Thanks!

Discord Support
Join Easy Multi Save Discord

Email Support
HegiDev@gmail.com

Update Changelog

Version 1.85

New Features
- Added 'Copy Save Slot' to advanced nodes
- Added 'Transfer Custom Object' to advanced nodes
- Added source and target slot support to cloud nodes

Improvements & Refactors
- Runtime Actors now call 'Actor Pre Load' before construction
- Plugin Settings now have their own category
- Improved cloud import/export file validation
- Improved Player verbose logs
- Improved low-level backup security
- Various small improvements

Bug Fixes
- Fixed 'Is Level Streaming' remaining true after cloud import
- Fixed Sub-Level Blueprint not loading with automatic stream load
- Fixed edge case where Reset Custom Save Object can clear its file name

---

Version 1.84

New Features
- Added 'On Game Actors Saved' delegate with convenient outputs
- Synchronous checks now work while game is paused

Improvements & Refactors
- Improved async task check logs and consistency
- Improved rare edge case check for imported cloud data
- Merged Streaming/World-Partition saving into one plugin setting
- Removed obsolete 'World Partition Saving' from settings
- Custom Player save now does not interfere with Multi-Level player data
- Various small improvements

API Changes
- Renamed the C++ async task checks. 'IsAsyncSaveOrLoadTaskActive' is now 'IsAsyncTaskActiveWithPause', with a new pause-independent 'IsAsyncTaskActive'.
  Blueprint nodes are unchanged.

- Streaming/World-Partition now always saves to memory only automatically.
  This keeps it consistent with other save operations,
  where data is saved to disk using 'Save Game Actors'

Bug Fixes

- Fixed edge case where stream data could not persist when save was canceled

---

Legacy Versions 1.0 - 1.84

The full changelog for older versions is available on Discord.

Example Project Version 2.8

- Added Actor Spawner Example Level
- Added Actor Reference Example Level
- Updated EMS_FunctionOverview Blueprint

Platform Information

Windows, PC VR
All features tested and working.

Nintendo Switch
XBox
Playstation
All features tested and working with some general restrictions. See Consoles.

Mac
All features tested and working.

iOS
All features tested and working.
Deleting save files might require additional permissions.

Android, Mobile VR
All features tested and working. See Android.

Unreal Engine 6

UE6 introduces major changes with Verse and Scene Graph replacing the traditional Actor and Blueprint framework. How the new framework will handle local save data remains to be seen. Despite the increased focus on online experiences, local saves are expected to remain just as important as they are today.

Once UE6 enters Early Access, we'll explore the possibilities for adapting EMS while preserving its core features, including open-world saving, save slots, custom saves, and reliable file-based persistence.

There are no firm commitments yet, and UE5 will continue to be fully supported.

Known Issues

All known issues will always be fixed in the next update.

⚠️With Skip Initial Load enabled, saving a new game may trigger a World Partition load warning and not capture World Partition cells as they unload until the first save.
Workaround: Call Load Game Actors *once with the Level flag selected. This initializes persistence even without save data. The Failed output is expected in that case.*

Credits

Easy Multi Save © by Michael Hegemann

Special Thanks
Rune de Groot, Jaden Evanger, Rama, JimW, Orfeas Eleftheriou, Muit, Matt Stafford, fgolge, Matthias Hölzl, Sukrit Sarankawin, Viktor Lidäng, Herr Edgy, Alan Edwardes, Wertix, Sid Spacewalker, Tuxedo Cat, lvhuming, mss10001, E_PepeGamer, Kamron, sp1r1t_d1tch, Lionard, Fabian Randau, sk83rjosh, TitamFR, Jorm, BeryMery, _pablo, Moonville Entertainment